Composite Design Pattern (CDP)

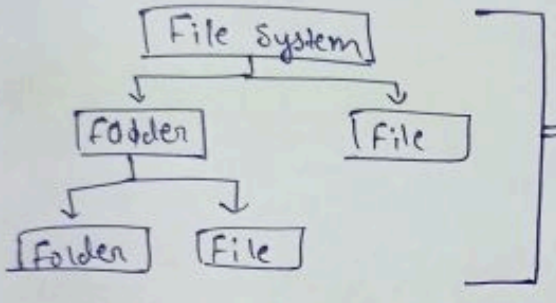
→ used in specific problem.

→ Composite pattern composes object into tree like structure representing a part - whole hierarchy. It let client treats individual object & composition of object uniformly.

* Introduction

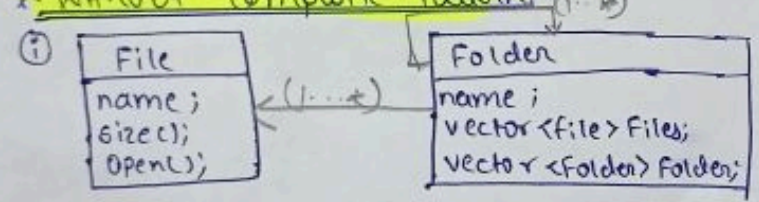
- Consider DSA concept using tree $\xrightarrow{\text{is similar}}$ to C.D.P.
- Tree have → root, node, leaf

Eg. File System



⇒ Tree like structure, ① File is the leaf node.
or
CDP e.g.
② Folder is the Non-leaf node also called composite node.

* Without Composite Pattern

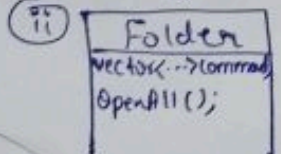


- Folder can have multiple file or folder too.

* Implementation hard ?

→ Commands → ① ls = list directory (list down all the file & folders) (But does not show inside content)
(No order preserved)
② OpenAll (Expanded form listing till leaf node.)

→ Opening



- If we want to open all the folder & file using OpenAll() then we can used single list or in cpp used pointer. To maintain order preserved.

Command List Looping ⇒

```
for (---) {
    if (element == file)
        cout << element->name;
    else
        element -> OpenAll();
}
```

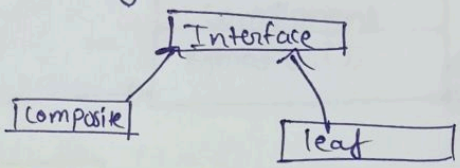
Problem ⇒ ① Treat as single object → implementⁿ of command is difficult.

② Treat as different object → Listing down sequentially Not possible.

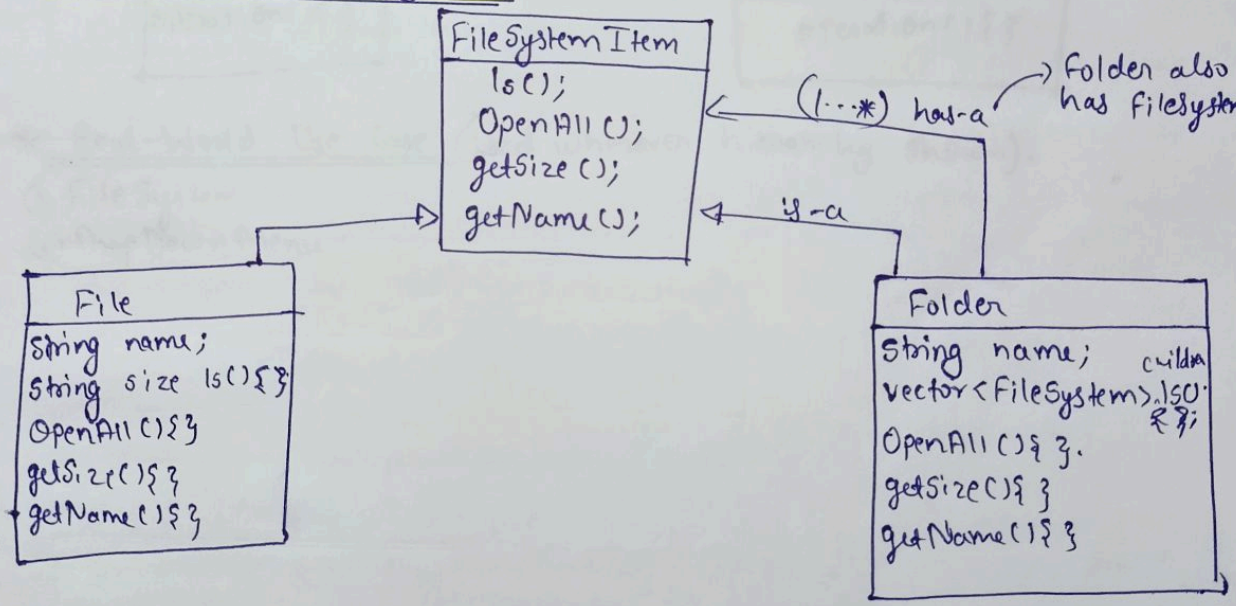
* What is composite pattern all about ?

↳ We have 2 types of element ↔ leaf, composite

↳ To yeh pattern kehta hai unn dono ke same tareeke se treat karo & unka ek common interface hoga jiske methods yeh override karega.



* UML for file system



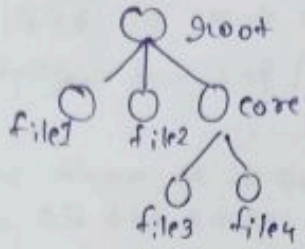
(+) root
- ↳ file 1.txt
- ↳ file 2.txt
- (+) core
  - ↳ file 3.txt
  - ↳ file 4.txt
- (+) user
  - ↳ file 5.txt

- If we call OpenAll() in root

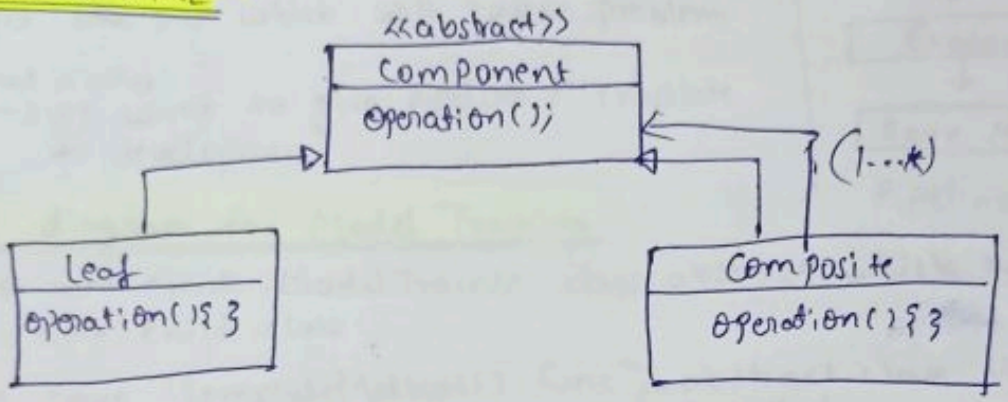
```
for (FileSystemItem : children) {
        cout<<Item->OpenAll();
}
```

- If item is file ⇒ OpenAll(){ getName; }

- If item is folder, recursive calls take place. It again loop through list found item if file → getName, if folder again recursion



* Standard UML



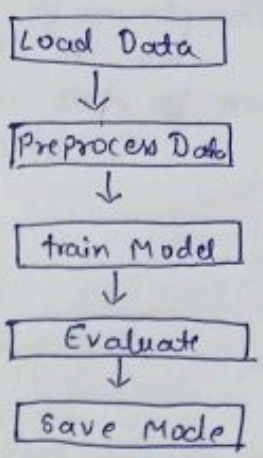
* Real-World Use Case. (used wherever hierarchy shown).

① File System
② Drop Down Menu

Template Method Pattern

* Introduction

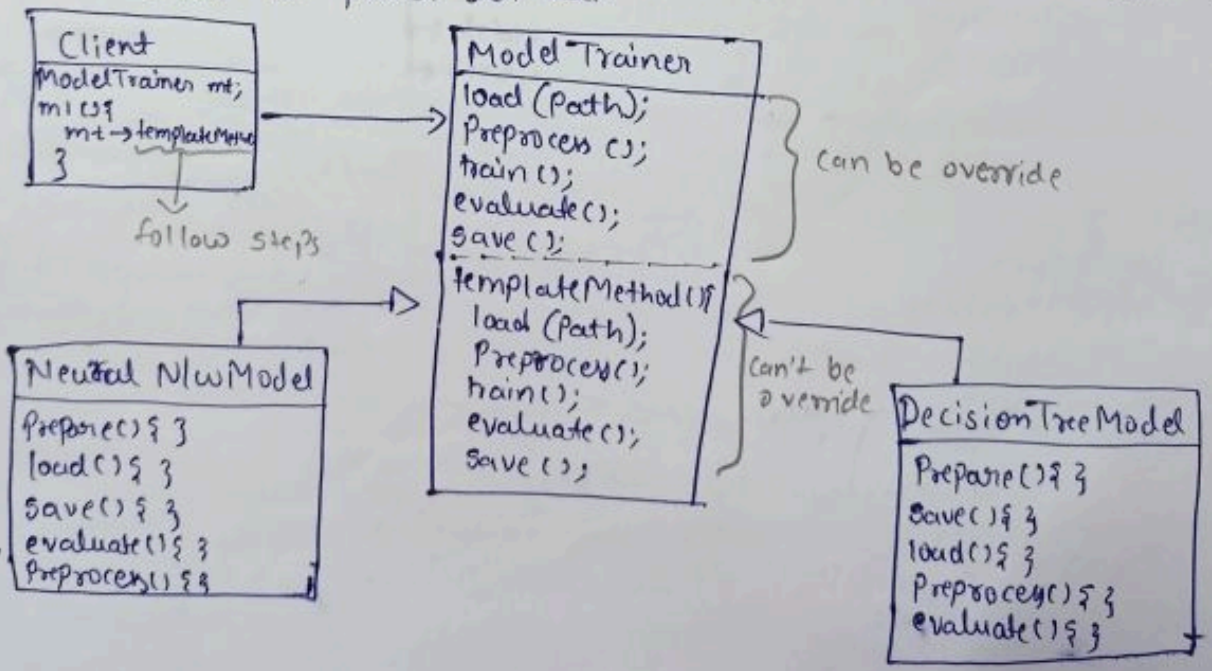
- There is pipeline (Rules Listed) to create Model
- It is important to follow this step / Rules one-by-one.
- If there is no pipeline than it is our responsibility to follow all the steps.
- There might be chances that developer forget one step which will cause problems
  - that is why
    - → we want to give pipeline / Template to developers.



Pipeline

* UML diagram for Model Training

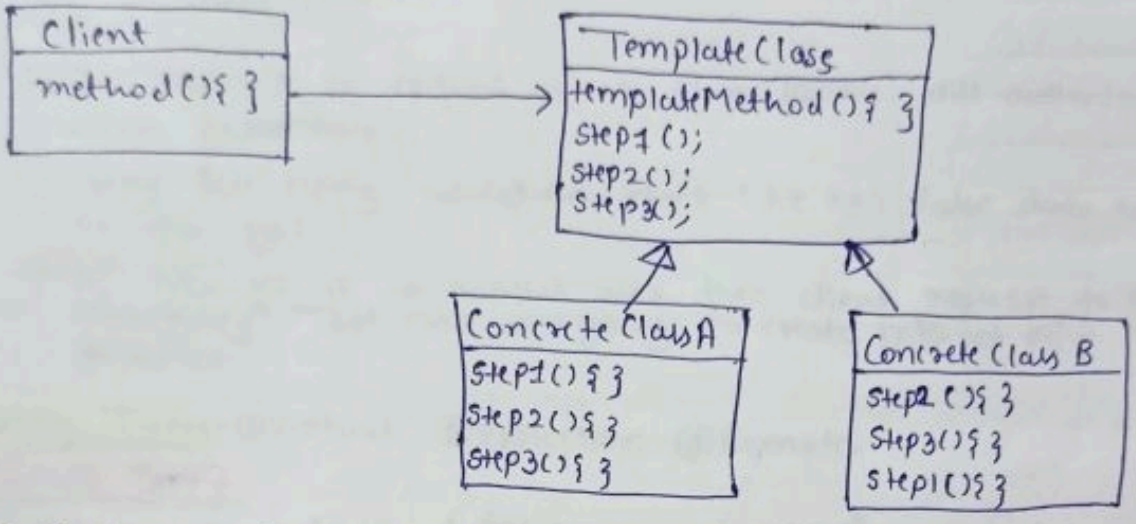
- → First we create ModelTrainer class abstract which have multiple child class.
- → We have TemplateMethod() func^{n}; in abstract class it will define the order in which it should called.
- → TemplateMethod() will not be ~~over~~ override by any child class bcz we will use const (cpp).
- → Whenever client calls, templateMethod() toh koi bhi order mein child class ne usse implement kiya ho par execution uska declared sequence mein hi hoga.
- → Template Method humein order of execu^{n} ko constant / same rakhne ki power deti hai.



* Standard definition

Template Method pattern design the skeleton of an algorithm for an algorithm operaⁿ defining some steps to subclass. Template method let subclass redefine certain steps of an algorithm w/o changing the algorithm structure.

* Standard UML dia.



* Real-World Use Case.

Used in scenarios where we want to follow order/sequence of execution.

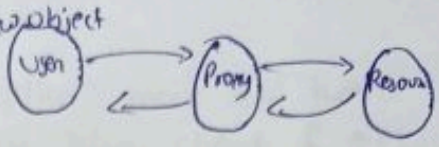
Eg. UPI payment → Validate balance
→ amount
→ pin
→ debit
→ credit.

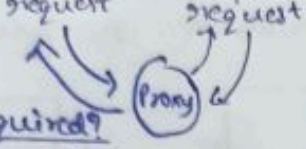
Proxy Design pattern

↳ representative of resource.

Introduction

- Used in specific problem.
- Consider, user & resource using new object





*** Why required?**

↳ ① If resource is critical object than proxy will authenticate user before requesting.

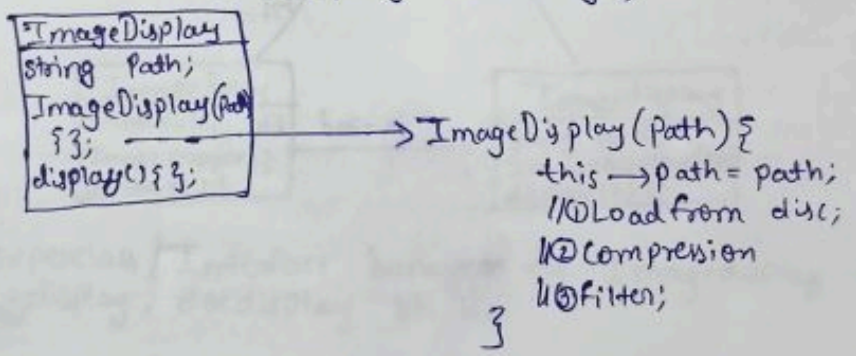
② Proxy can apply validation check like koi false data toh nhi aa rha hai.

③ If resource is in another area then client request to some class (proxy). That class will going to create internet n/w to connect resource.

*** Proxy Types:-** ① Virtual ② Protection ③ Remote.

① Virtual Proxy

↳ Take e.g. to understand. ("Display" an Image")



- Ab jab client display() call karega toh image display hoga according to path specified.
- Par load karne ke phele usse kuch task perform karne padege joh humne pass kiye hai constructor mein.
- If we pass all this task in display(){} so it take time to display image so we pass it in constructor.

Client code → `ImageDisplay * img = new ImageDisplay(p);`

`img → display();`

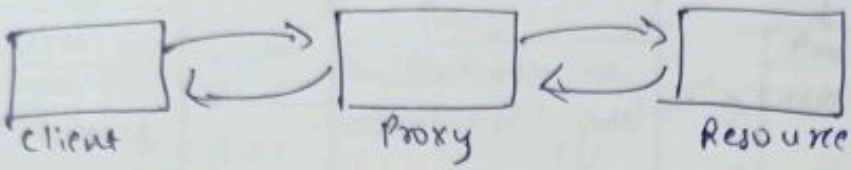
It will perform expensive operation

Client calls display method.

Problem

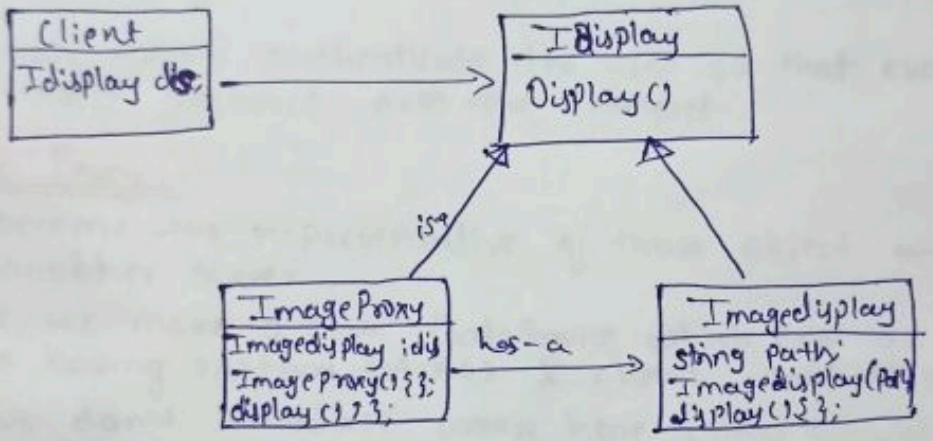
*What if client doesn't call display method?
↳Object toh create hoga server par & expensive operⁿ ke karan time bhi waste hoga.

→Yeh bas eg hai but large applicⁿ mein bhi hota hai aisa.
→That's why we introduced proxy b/w client & resource.



- Proxy introduce karne se ab proxy reference hold karega ImageDisplay ka & directly object nhi banayega. It will only create object when client calls display method.

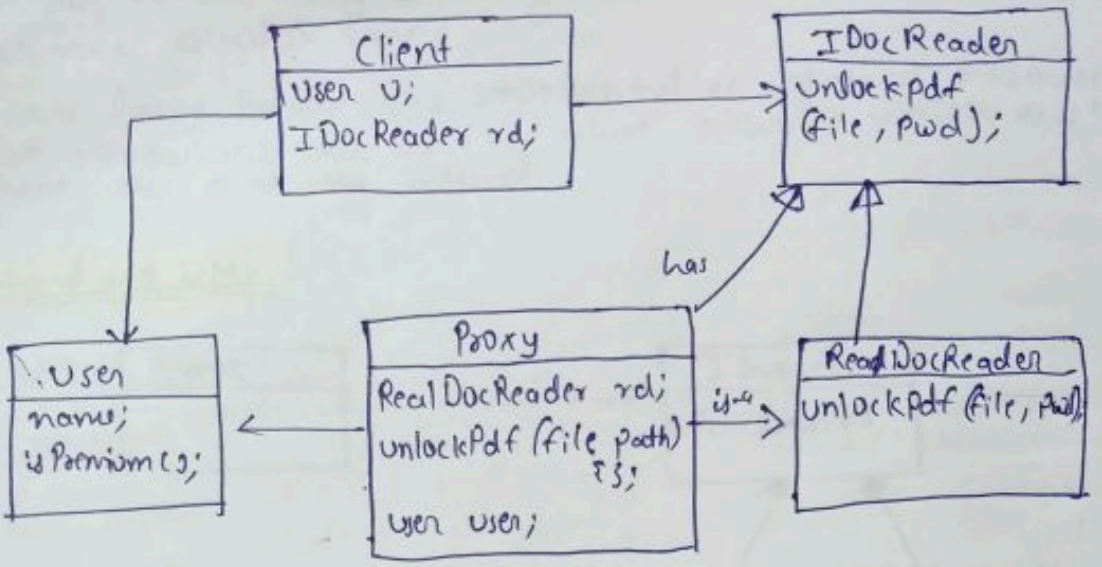
*UML diagram



- Phele hum superclass / Interface banayege for Imagedisplay taki future mein vdisplay, docdisplay bhi ho.
- Proxy will have reference of ImageDisplay & proxy will behave like image display.
- Client call karega IDisplay ko, client ko nhi pata hoga that what we have passed ImageProxy or Imagedisplay but we will always pass Imageproxy.

② Protection Proxy

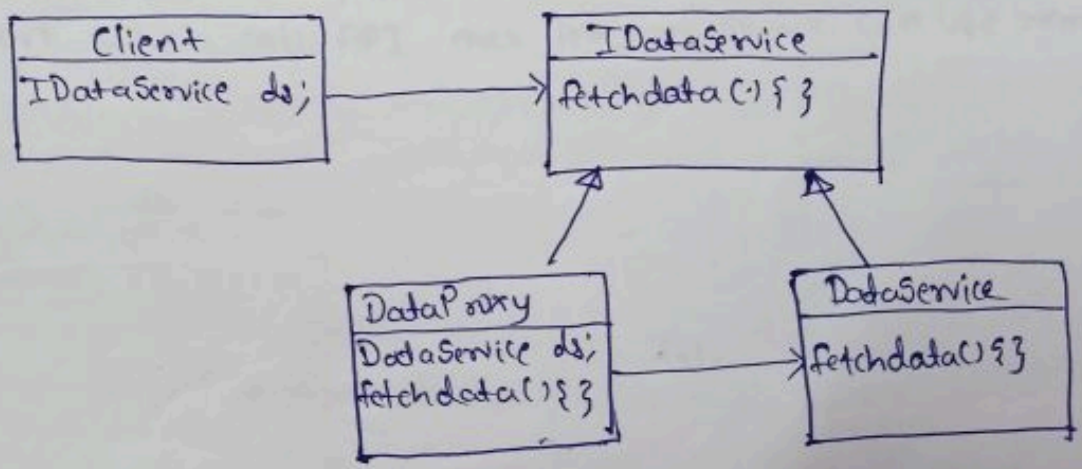
- Let's understand with e.g. of DocReader.
- We create one class which have feature to unlock pdf which will be limited to premium user.



- When client call unlock pdf then request goes to proxy & in proxy first it check whether the user is premium or not. If user is premium only then it will call unlock pdf() else throw error.
- Protection proxy authenticate the user so that everyone can't access those resource over the internet.

③ Remote Proxy.

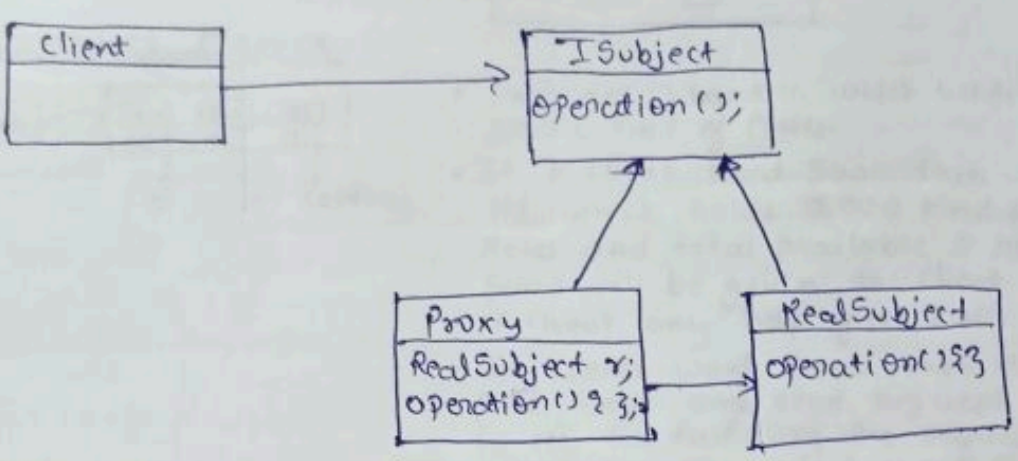
- It become the representative of those object which exist on another server.
- there, we make a class DataService which exist on another server having method fetch() & client want to use this method.
- If we don't introduce proxy here client should know about server & then establish connection to call this method.
- So, we introduce DataProxy bcz we don't want that client should know about server & all.



Here, we have two ways for establishing connect:
↳ ① When client create object => ds → new DataProxy; //first loading
② When client call fetchdata() => ds → fetchdata(); //lazy loading

- Here we follow lazy loading when client call method only then we establish conn^n.
- Remote Proxy become the representative of that resource which exist somewhere else over of that resource which exist somewhere else over the internet.

*Standard UML



*Standard definition

"The proxy pattern provides a surrogate or placeholder another object to control access to it.

- Remote Proxy
- Virtual "
- Protection "

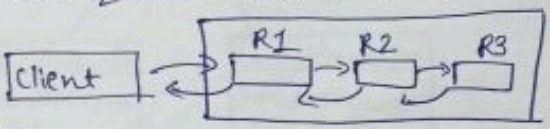
*Real World Usage

① User authenticate for premium feature.
② Framework which call API over internet so we can use remote.

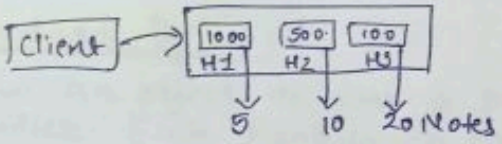
Chain of Responsibility Pattern

*Introduction

- Client sends request a group of object. Like client sends request to R1 object, if R1 fulfill the request than it respond to Client directly. If it does not fulfill the request than it sends to R2 and so on. (like a linked list).

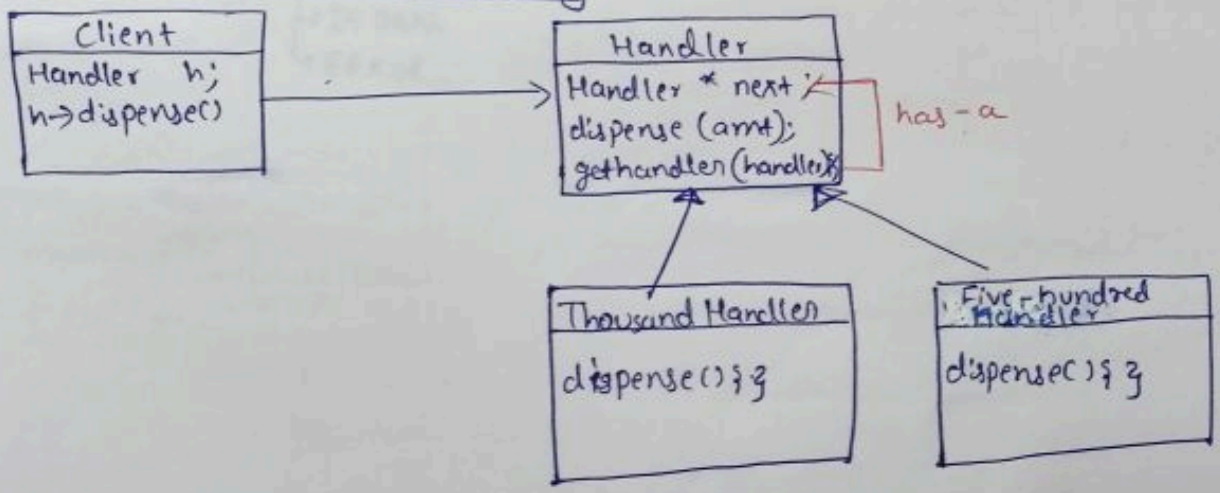


E.g. Cash Dispensing Machine.

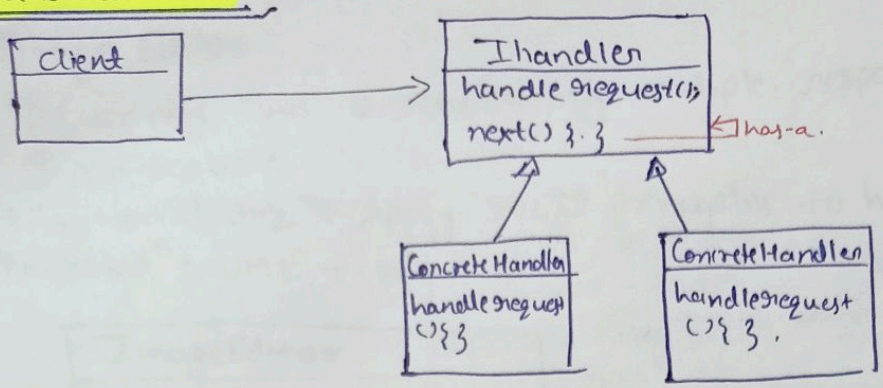


- There are 3 handler which holds the specific kind of Notes.
- If a client want 5000 then H1 which holds 1000 kind of Notes and total available 5 means 5000 will be given to client without any help of H2 & H3.
- If clients want 6000 then H1 give 5000 and send request to H2 for fulfilling the request remaining. Then H2 will see, that if it is fulfilling the request or not. If yes then No further request send. In our case, Yes, and it will give 500 x2 Notes.

*UML dia. for cash Dispensing.



* Standard UML



* Standard Defn:-

Allow an object to pass a request along a chain of potential Handlers. Each handler in the chain decides either to process the request or pass it to the next handlers.

* Difference blw Linked List & COR.

- It stores data → It contain other object bcz if it does not fulfill then it will pass
- 1 Class have different object → different class are linked different object too.

* Real-World use case.

logger System →
- INFO
- DEBUG
- ERROR.

Problem Statement

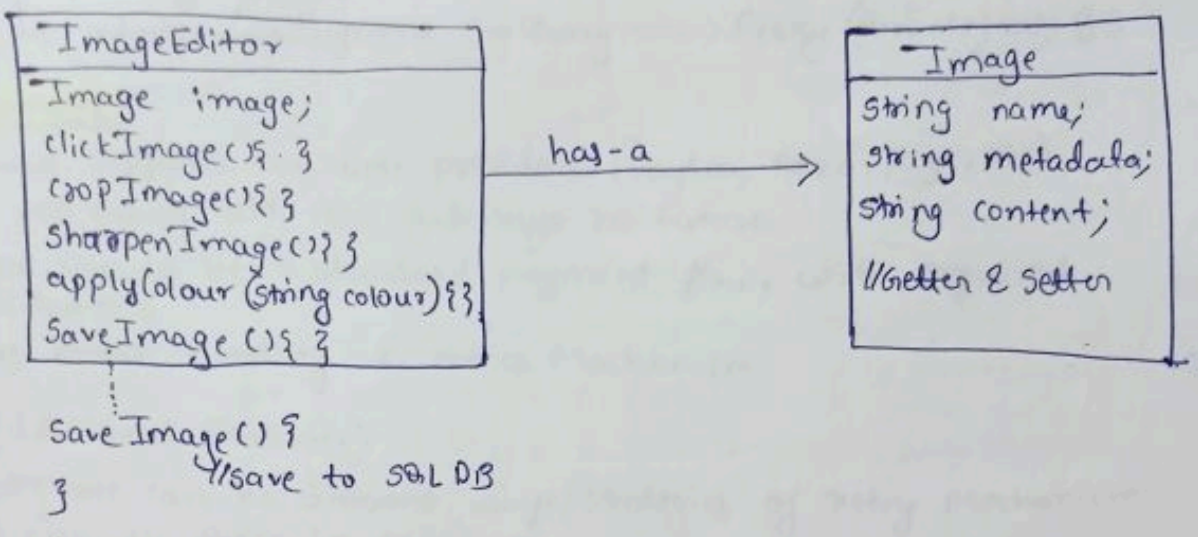
1) Image Editor

→ (i) Currently class is maintaining mutiple responsibilities.

(ii) Not scalable

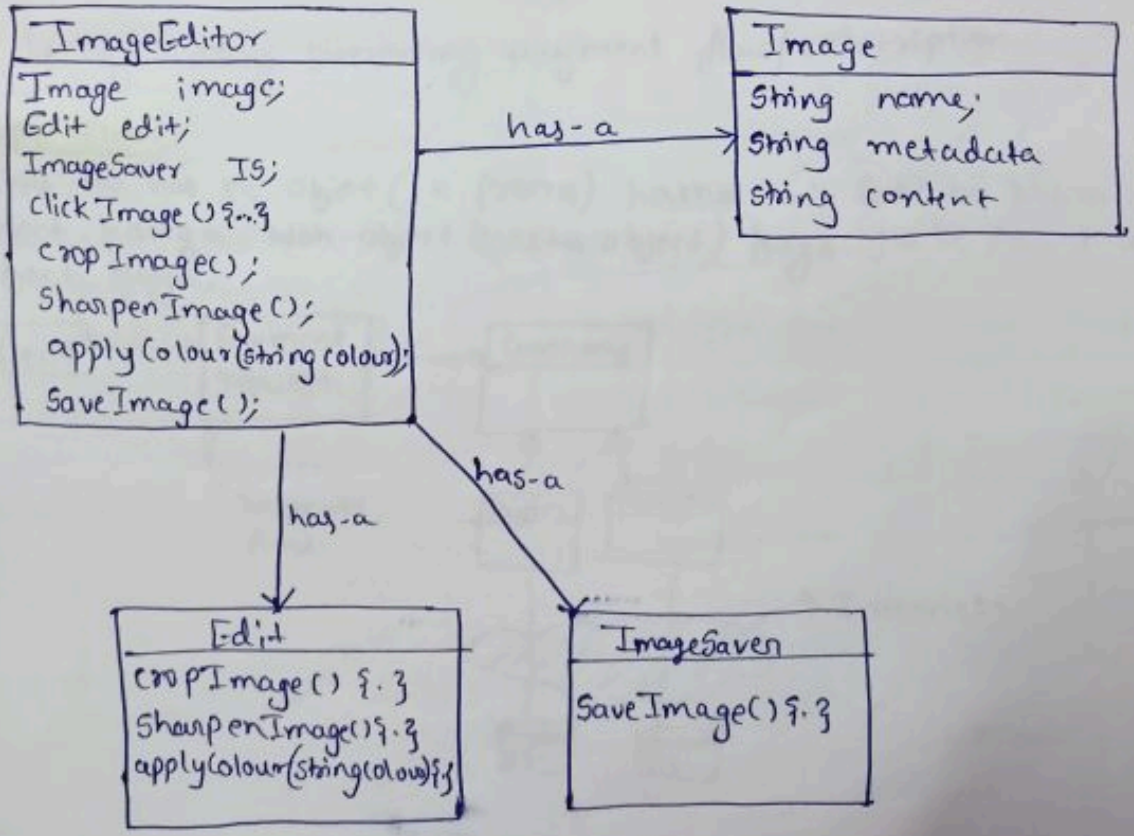
→ Functional req.ⁿ :- Apply SOLID principles to handle the above problem

→ Expectatⁿ : UML + code



Solution:

- UML dia



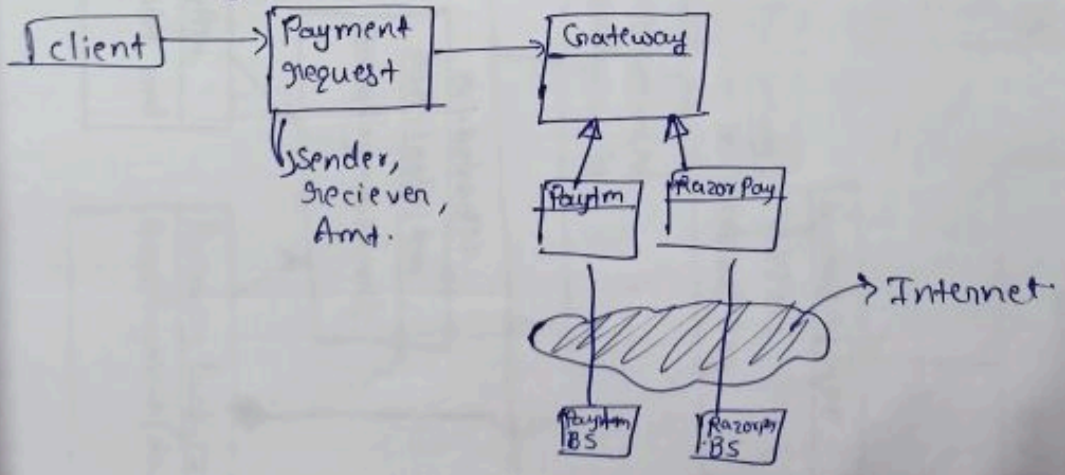
Building Payment Gateway System

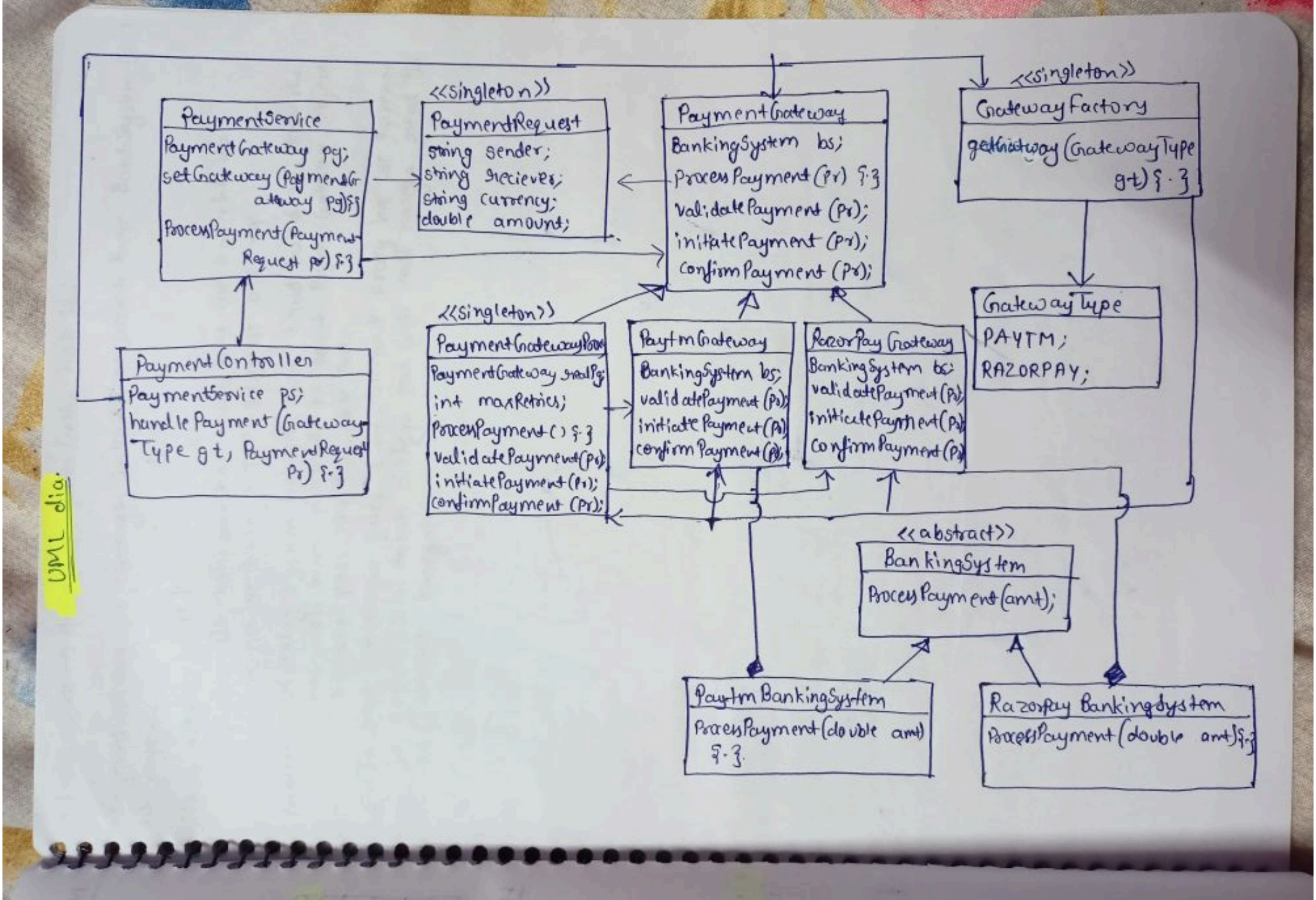
Noted:- ① We are building payment Gateway
② We are not building Banking System (CRUD Operation)
- Basically jaha par transaction process ho rha hai Hum bas integrate kar rhe hai Banking System ko jis mein hum client ko ek mediator ke through interact karwayenge Banking System se.
- Mediator ⟹ Payment Gateway ⟹ Proxy (Remote) ⟹ BS.

Requirement
- Should supports multiple providers. (Paytm, Razorpay, etc).
- We can easily add new gateways in future.
- There should be a standard payment flow, with required validations.
- Have error Handling & Retries Machanism.

H.W (Additional Features)
- There can be different ways/strategies of retry Mechanism (Handle them in code).
  - Linear ~~Handle~~ Retry
  - Exponential Back-off.
- Try adding recurring payment flow/subscription.

* Happy flow
- Client ko bas ek object (ek process) karna hai. Baki ka kaam woh object karega. Woh object (master object) hoga. jis ke pass multiple object hoga.




UML dia.
PaymentService
PaymentGateway pg;
setGateway(PaymentGateway pg){}
ProcessPayment(PaymentRequest pr){}
<<Singleton>>
PaymentRequest
string sender;
string recieverId;
string currency;
double amount;
PaymentGateway
BankingSystem bs;
ProcessPayment(Pr){}
validatePayment(Pr);
initiatePayment(Pr);
confirmPayment(Pr);
<<singleton>>
GatewayFactory
getGateway(GatewayType gt){}
PaymentController
PaymentService PS;
handlePayment(GatewayType gt, PaymentRequest Pr){}
<<Singleton>>
PaymentGatewayProxy
PaymentGateway realPg;
int maxRetries;
ProcessPayment(){}
validatePayment(Pr);
initiatePayment(Pr);
confirmPayment(Pr);
PaytmGateway
BankingSystem bs;
validatePayment(Pr);
initiatePayment(Pr);
confirmPayment(Pr);
RazorPayGateway
BankingSystem bs;
validatePayment(Pr);
initiatePayment(Pr);
confirmPayment(Pr);
GatewayType
PAYTM;
RAZORPAY;
<<abstract>>
BankingSystem
ProcessPayment(amt);
PaytmBankingSystem
ProcessPayment(double amt){}
RazorpayBankingSystem
ProcessPayment(double amt){}

* Model class ⇒ basic class, getter/setter hote hai.

• PaymentGateway ke through internet se juzarte huye BankSystem (BS) tak jayega.

Now, why internet?

↳ Bcz BS. kahi aur hai. Hum bas use kar rhe hai.
↳ internet se hum HTTP request bhejenge.

Problem :- BS. ka alag system hai joh hum khud build nhi kar skte hai. Toh hum normal ki tarah PaymentGateway Object banake pass nhi kar skte hai.

Sol" :-> In real - interview ques" time, remote proxy ka use karenge jo same BS ki tarah dikhega par woh call karega real BS ko (interact karega).

Build Discount Coupon Engine.

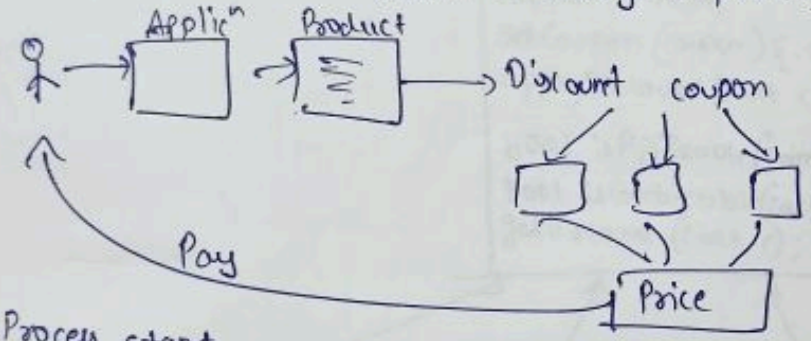
↳ Plug-&-play

* Functional requirement.

- We can all new coupons at Runtime.
- Both cart level & product level discounts.
- Supports Multiple coupons types → Premium remember only
  (Seasonal offers, loyality discounts, Banking discount etc).
- Supports both flat & percentage discount.
- One coupon can/cannot be applied on top of other coupons.
- Thread safe :- C++ ⇒ mutex used, Java ⇒ semaphore,
  ↳ Consider we have 4 same type of coupon. So, will be used thread

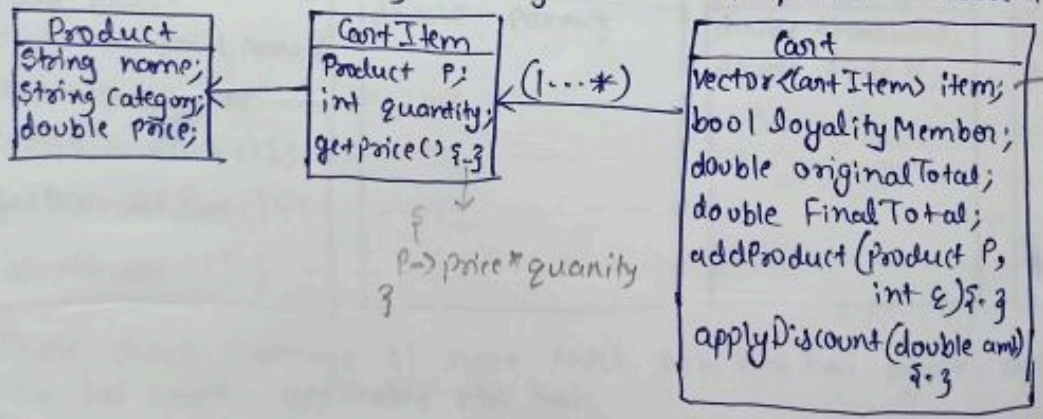
* Happy flow

Always remember ⟹ discount/coupon will be applied on product.
We are not mention product name bcz
it will be change depending on application-to-applica

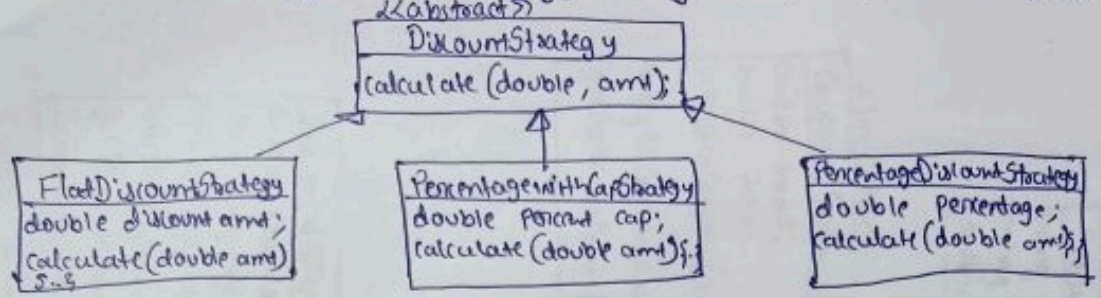


* Process start

We will take inventary Management like zepto/Blinkit have it.



Discount strategy with strategy design pattern (can be replacable).

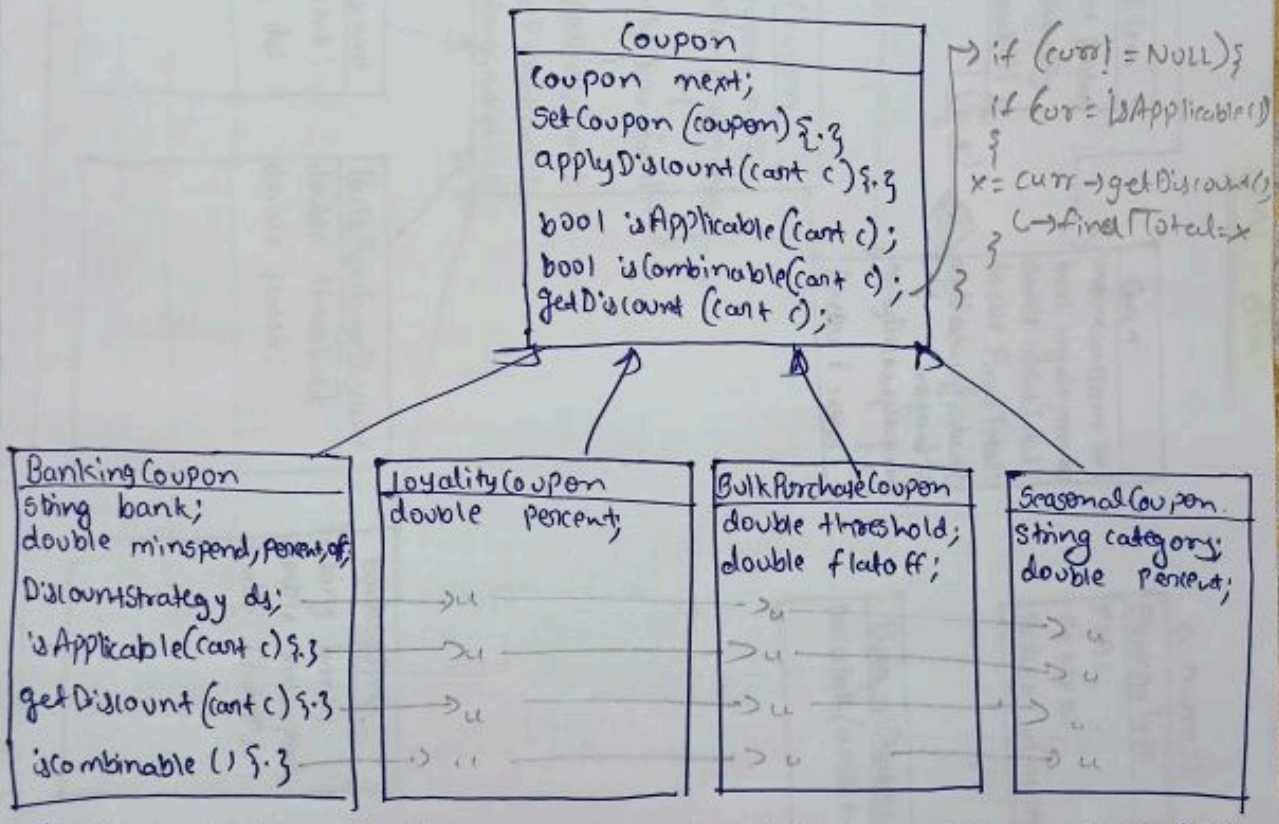


* Apply different types of coupon.

↳ coupon class ⇒ • It have next variable (holds the next coupon) (we are using COR pattern here we can also use decorator pattern).

• SetCoupon() ⇒ will have next variable (has-a r/nship).

• applyDiscount (cart c) {...} ⇒ It will call to cart method.

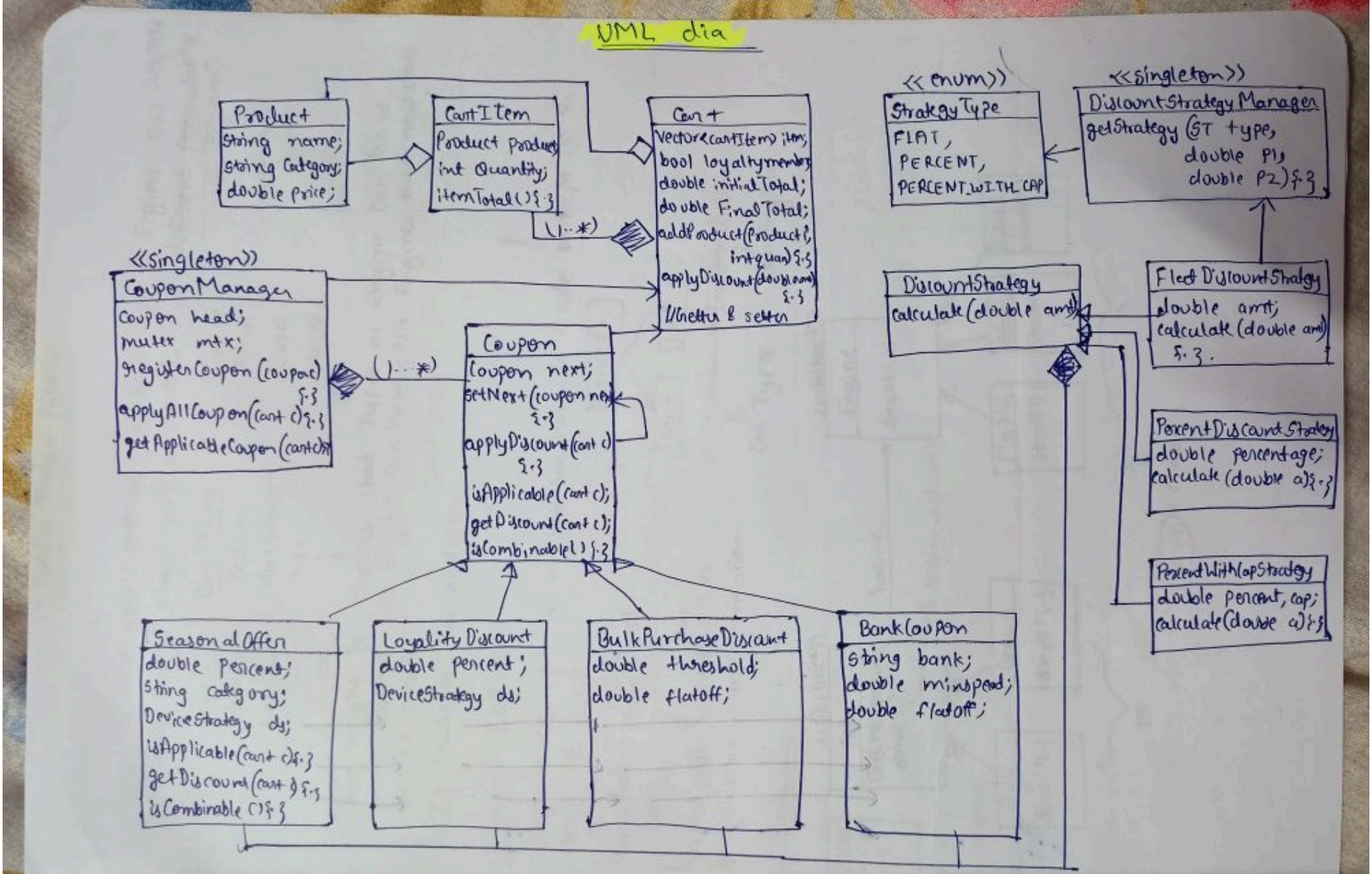


→ Phele check karenge ki next NULL toh nhi hai kyuki agar NULL hai toh koi coupon applicable nhi hai.

→ Then isApplicable() call hoga kyuki joh coupon avaliable hai woh applicable hai bhi yaa nhi.

→ Agar applicable hai toh getDiscount() call hoga.

→ isCombinable() kisi-kisi coupon par hi available hoga.


UML dia
Product
string name;
string Category;
double price;
CartItem
Product product;
int Quantity;
itemTotal(){}
(1..*)
Cart
Vector<cartItem> item;
bool loyaltymember;
double initialTotal;
double FinalTotal;
addProduct(Product*, int quan){}
applyDiscount(double amt){}
//Getter & setter
<<enum>>
StrategyType
FLAT,
PERCENT,
PERCENT_WITH_CAP
<<singleton>>
DiscountStrategyManager
getStrategy(ST type, double p1, double p2){}
<<Singleton>>
CouponManager
Coupon head;
mutex mtx;
registerCoupon(Coupon*){}
applyAllCoupon(Cart c){}
getApplicableCoupon(Cart c)
(1..*)
Coupon
Coupon next;
setNext(Coupon next){}
applyDiscount(Cart c){}
isApplicable(Cart c);
getDiscount(Cart c);
isCombinable(){}
DiscountStrategy
calculate(double amt)
FlatDiscountStrategy
double amt;
calculate(double amt){}
PercentDiscountStrategy
double percentage;
calculate(double a){}
PercentWithCapStrategy
double percent, cap;
calculate(double a){}
SeasonalOffer
double percent;
string category;
DeviceStrategy ds;
isApplicable(Cart c){}
getDiscount(Cart c){}
isCombinable(){}
LoyalityDiscount
double percent;
DeviceStrategy ds;
BulkPurchaseDiscount
double threshold;
double flatoff;
BankCoupon
string bank;
double minspend;
double flatoff;

Bridge Design Pattern

*Introduction

→ Consider, car have drive() method. Car have Types & Engine 3,3 respectively
→ It means 3x3 = 9 classes will be created, with different permutat^n & combina^n.

Car
drive();

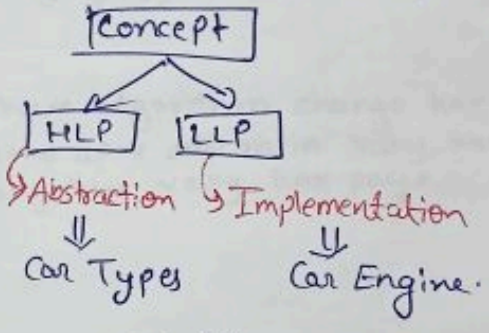
Car Types
Sedan
Hatchback
SUV
⋮

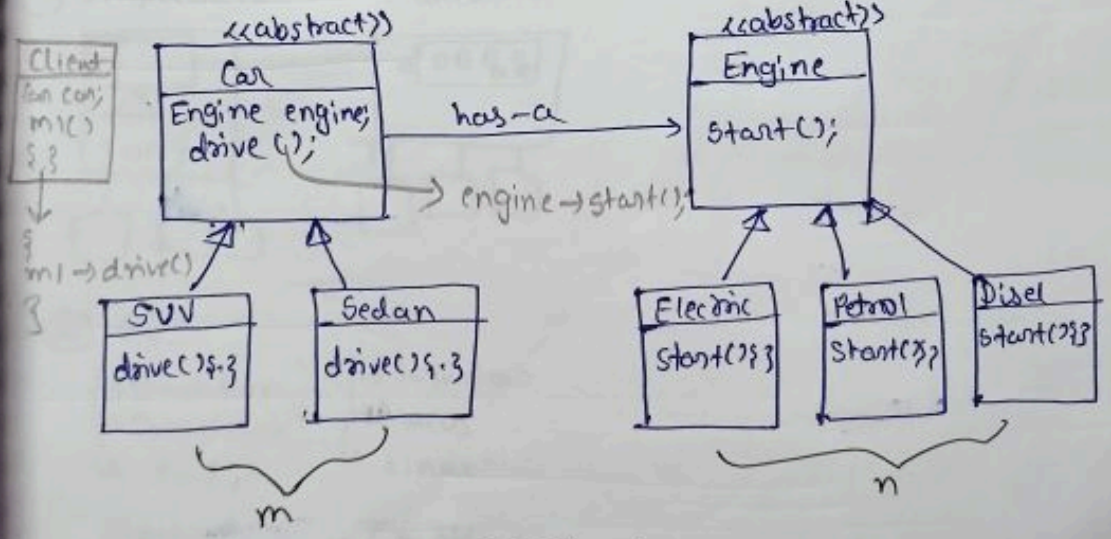
Engine
Petrol
Diesel
Electric
⋮

→ There might be chances that Types or Engine can have more features like Engine can have CNG Engine, than new class will be create.
→ It leads to class explosion.

Solution:- BDP used.

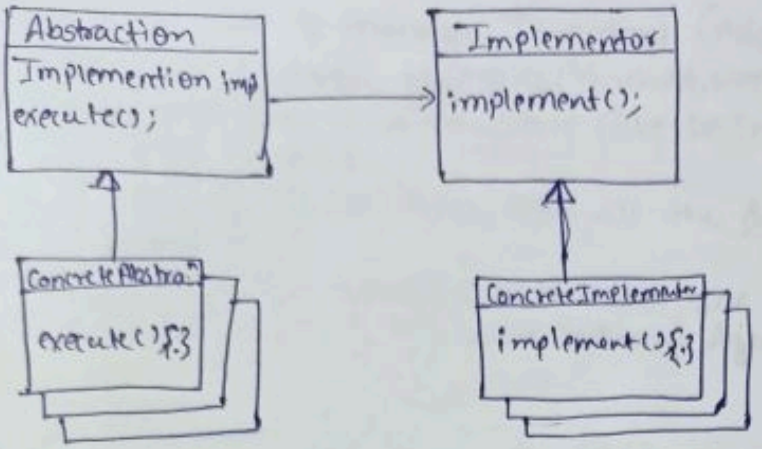
Consider concept is a class. We have to divide concept in 2.
HLP & LLP
⇓ ⇓
High Level Part → What will be architecture
Low Level Part → What/How will be implementation.





(m+n) = class.

* Standard UML diagram.



* Standard definition

↳ Bridge decouples an abstractions from its implementation, so that both can vary independently.

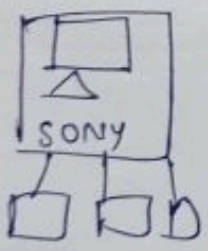
- Abstractions: High level layer. (car).
- Implementations: Low Level layer (Engine).

* Strategy & Bridge difference.

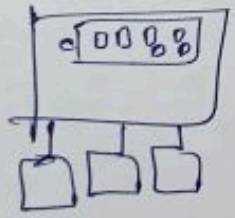
* Intent: Strategy ⇒ dynamically hum algorithm change kar skte hai.
Bridge ⇒ HLP & LLP, dono apne aap mein vary kar paye.
Dono interchangably vary kar paye.

* Real-World use case.

① Implement



Abstraction



② GUI
↳ checkbox
↳ Dropdown
↳ Radio

Abstra^n

OS
↳ Window
↳ MacOS
↳ Linux.

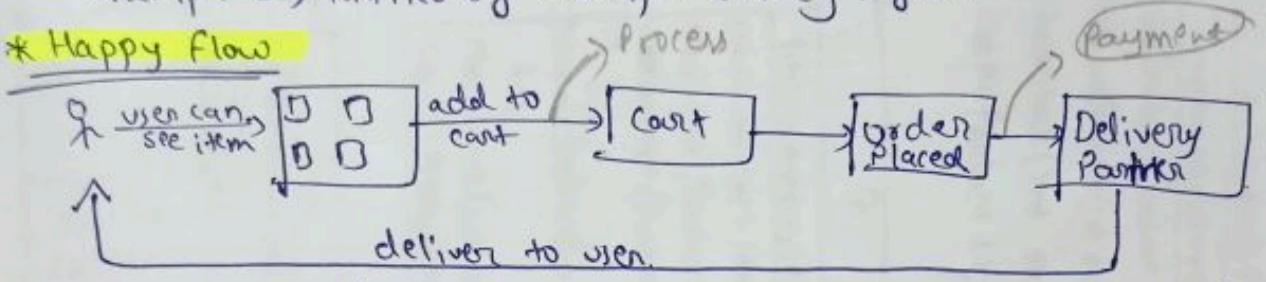
Implementation

Build Zepto (Inventory Management)

* Requirement

- We should be able to manage Inventory (Add / remove item).
- We should have replenish strategies (Threshold, weekly) & should be scalable.
- We can have multiple InventoryStore (like DBInventoryStore etc) & we can further extended.
- User should able to see items from all the darkStores closer to him / her (5 km).
  - darkStores → Inventory rakhte hai yahi
- If one DarkStore cannot fulfil order, one order can be split into multiple DS, fulfiled by multiple Delivery Agents

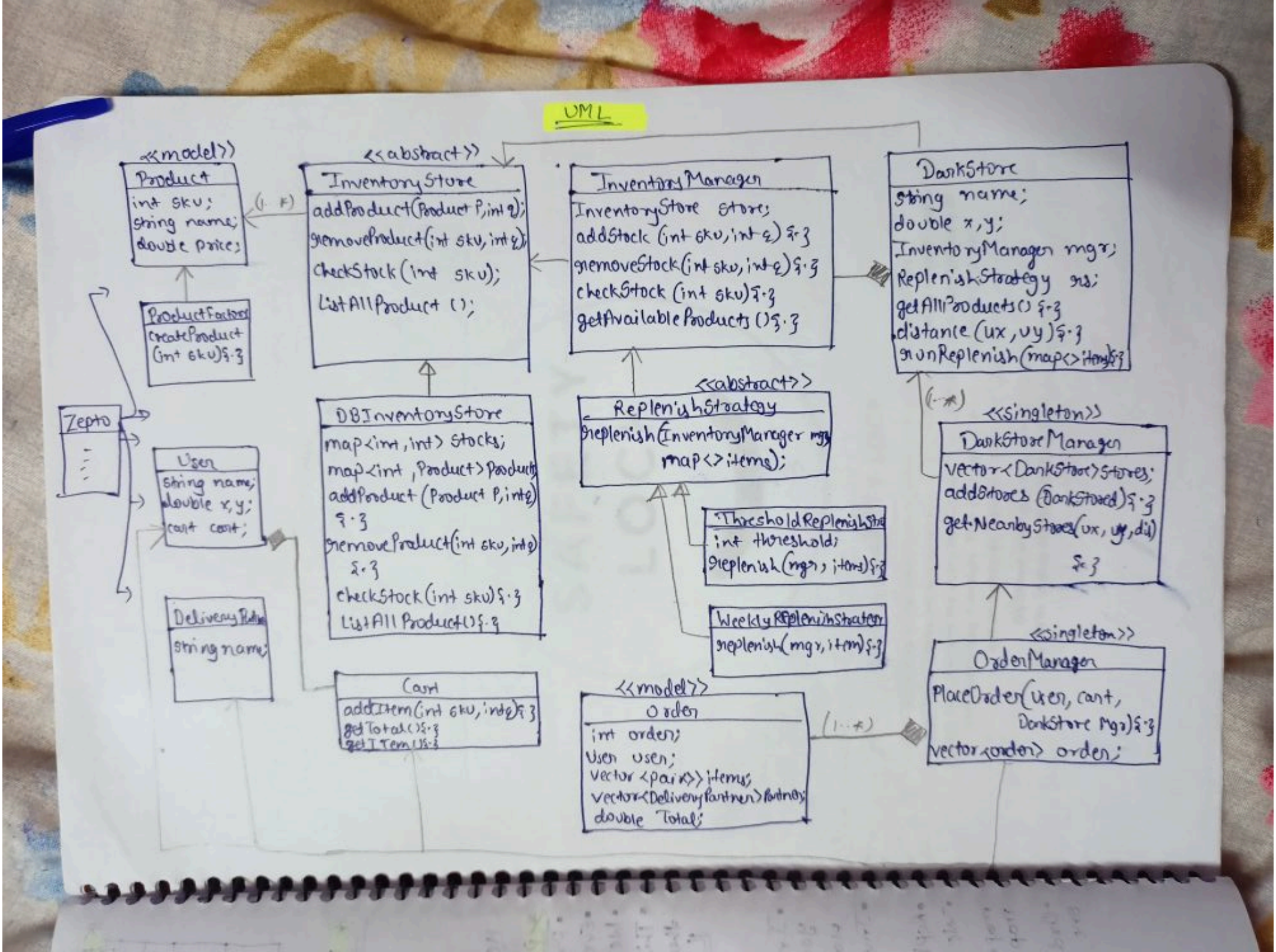
* Happy Flow



Here, we are not discussing payment method bcz we had already done.

* Process

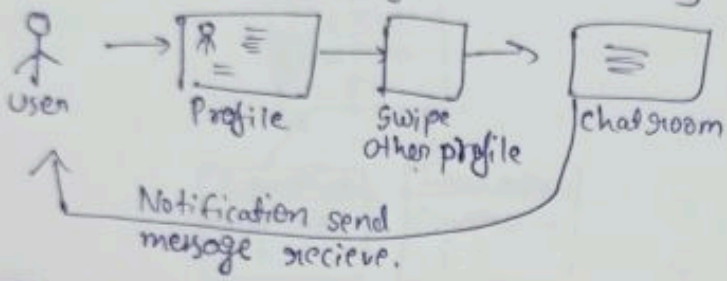
- Product id (SKU). ProductFactory will create product (how? → by product id).
- Every darkstore have 1 InventoryStore. InventoryStore will store multiple product
- We will have manager between InventoryStore & DarkStore.
- This manager, InventoryManager will handles ~~how~~ create product (if required) then tell InventoryStore to add the product.
  - addStock (sku, q) f.e. → calls → ProductFactory → gives Product → then Manager → calls InventoryStore addProduct(-ff
- In InventoryManager, we have mostly same method as InventoryStore. you will be thinking that we are breaking DRY Rule. But we are not !!! How? ⇒ Manager only manages, tell others how to work.
- InventoryManager can be Singleton or not.
  - we have not → bcz Darkstore are on different place.
- ReplenishStrategy will have no. of item (that to be restore) + InventoryManager object
- We have to store somewhere ⇒ DB InventoryStore
  - map<int, int> stocks;
  - map<int, product> product
  - there might be chances that in future we will have file store or excel store. We use map instead of it.
  - So it will tell to add it on the stock.
- OrderManager manages the order. It will try to get all the product in one darkStore. But if it is not then it will divide order into 2 delivery agent


UML
<<model>>
Product
int sku;
string name;
double price;
<<abstract>>
InventoryStore
addProduct(Product P, int q);
removeProduct(int sku, int q);
checkStock(int sku);
ListAllProduct();
InventoryManager
InventoryStore store;
addStock(int sku, int q) {..}
removeStock(int sku, int q) {..}
checkStock(int sku) {..}
getAvailableProducts() {..}
DarkStore
string name;
double x, y;
InventoryManager mgr;
ReplenishStrategy rs;
getAllProducts() {..}
distance(ux, uy) {..}
runReplenish(map<>items);
(1..*)
ProductFactory
createProduct(int sku) {..}
Zepto
User
string name;
double x, y;
cart cart;
DBInventoryStore
map<int, int> stocks;
map<int, Product> products;
addProduct(Product P, int q) {..}
removeProduct(int sku, int q) {..}
checkStock(int sku) {..}
ListAllProduct() {..}
<<abstract>>
ReplenishStrategy
replenish(InventoryManager mgr, map<> items);
<<singleton>>
DarkStoreManager
vector<DarkStore> stores;
addStores(DarkStore d) {..}
getNearbyStores(ux, uy, dis) {..}
ThresholdReplenishStrategy
int threshold;
replenish(mgr, items) {..}
WeeklyReplenishStrategy
replenish(mgr, item) {..}
DeliveryPartner
string name;
Cart
addItem(int sku, int q) {..}
getTotal() {..}
getItem() {..}
<<model>>
Order
int order;
User user;
vector<pair<>> items;
vector<DeliveryPartner> partners;
double Total;
(1..*)
<<singleton>>
OrderManager
PlaceOrder(user, cart, DarkStore Mgr) {..}
vector<order> order;

Build Tinder

* Functional requirements:-
  - User can swipe left/right to a profile.
  - User can setup his/her own profile.
  - User can set his/her preferences.
  - Once there is a match they can chat in chatroom.
  - User can see all the profiles near their location. (Nearby can be based on different strategy)
  - User should get notified when there is a match, or recieve a new message.
  - User matching should be based on several factors & slow. (like interest match, location match, etc).

* Happy flow:-
  - It is the simplest system that is why we have simple Happy flow.



  - Top-down approach ⟹ We have multiple small components that is why Top-down approach.

* Process.

① Notification System (Observable pattern used).
  - Notifi^n Service ⇒ It is a Singleton class and we don't have many/one subclass bcz notification system/process will be same that is we'll send notification to 2 person when they are matched.
  - NotificationObserver ⇒ It have many subclass for now we have one. bcz notification sending process can be many in futture.

② User
  - (i) UserProfile ⇒ It will have basic details and Enum Gender class bcz in future we can add more gender. Interest class in which user will store his/her interest/hobbies. It will also have location of his/her so we can find people within them area (consider 5km)
  - (ii) UserPreference ⇒ In this class, user will decide what kind of person they are looking for by setting filters.
  - (iii) Swipe ⇒ It is a enum class which will provide to do left or right swiping.

- In user class we have swipe history basically provides us to know the history or record the profile which is swiped.
- We will also have boolean value for like & unliked the person or interested.

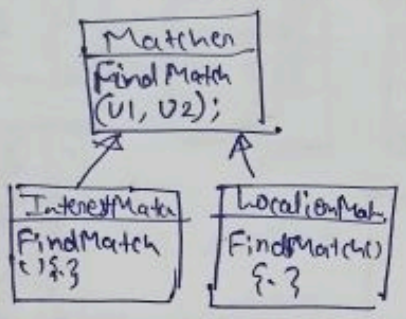
③ LocationService

↳ LocationService → This will going to take user location from location class and calculate how many users(others) are in 5km. It also have vector of user list.

↳ NearbyStrategy => It is a method.

④ Matching Algorithm

↳ Consider there is matcher class have concrete class like InterestMatcher & LocationMatcher



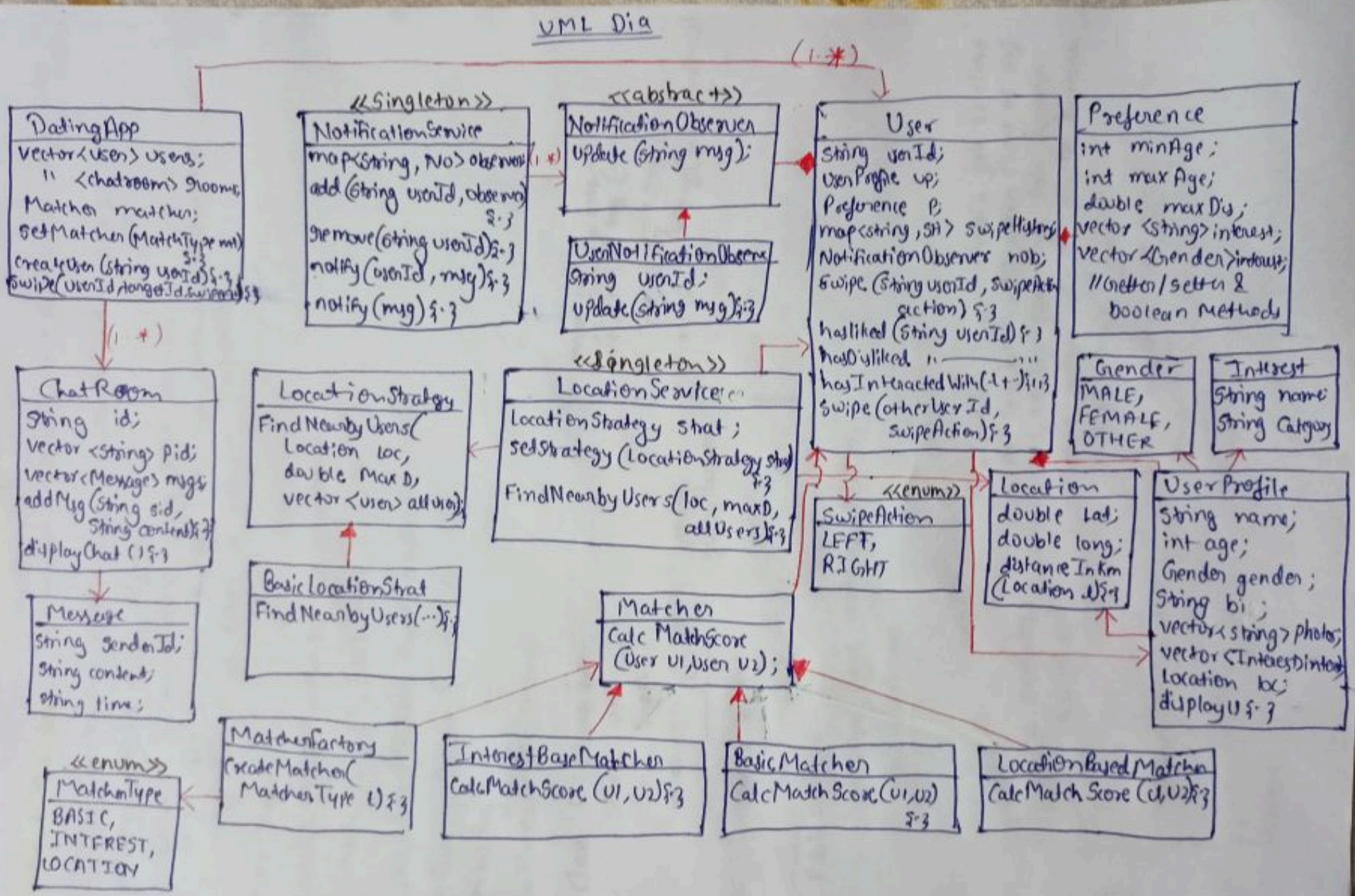
- We cannot decide Matching to profile by one situation.
- So, now a days we have different algorithm like by location, interest.
- We perform one algorithm then move to other it will go all the concrete class.
- At the end it will calculate over all score for Matching the 2 users.

We used COR

⑤ Chatroom

↳ It is class which have chatbot features Not AI.
2 person can talk, also can make group

UML Dia


(1 - *)
DatingApp
vector<User> users;
// <chatroom> groom;
Matcher matcher;
setMatcher(MatchType m);
createUser(string userId);
swipe(userId, targetId, swipeAction);
(1 - *)
<<Singleton>>
NotificationService
map<String, NO> observers
add(String userId, observer);
remove(String userId);
notify(userId, msg);
notify(msg);
(1 - *)
<<abstract>>
NotificationObserver
update(String msg);
UserNotificationObserver
String userId;
update(String msg);
User
String userId;
UserProfile up;
Preference p;
map<string, Si> swipeHistory;
NotificationObserver nob;
Swipe(String userId, SwipeAction);
hasLiked(String userId);
hasDisliked ...
hasInteractedWith(...);
swipe(otherUserId, swipeAction);
Preference
int minAge;
int maxAge;
double maxDis;
vector<String> interest;
vector<Gender> interest;
// Getter/setter & boolean methods
Gender
MALE,
FEMALE,
OTHER
Interest
String name
String Category
ChatRoom
String id;
vector<String> pid;
vector<Message> msgs;
addMsg(String sid, String content);
displayChat();
Message
String senderId;
String content;
string time;
LocationStrategy
FindNearbyUsers(
Location loc,
double maxD,
vector<user> allUsers);
BasicLocationStrat
FindNearbyUsers(...);
<<Singleton>>
LocationService
LocationStrategy strat;
setStrategy(LocationStrategy strat);
FindNearbyUsers(loc, maxD, allUsers);
<<enum>>
SwipeAction
LEFT,
RIGHT
Location
double lat;
double long;
distanceInKm(Location l);
UserProfile
String name;
int age;
Gender gender;
String bi;
vector<string> photo;
vector<InterestDinter>
location loc;
display();
Matcher
calcMatchScore(User u1, User u2);
MatcherFactory
createMatcher(MatcherType t);
<<enum>>
MatcherType
BASIC,
INTEREST,
LOCATION
InterestBaseMatcher
calcMatchScore(u1, u2);
BasicMatcher
calcMatchScore(u1, u2);
LocationBasedMatcher
calcMatchScore(u1, u2);

Builder Design pattern

↳ mostly used.
↳ whenever it comes to creating obj, we use builder design pattern.

*Without Builder

Problem 1: Constructor overloading (constructor Telescoping).

E.g. When client sends request.
↳ which go to server using http request.
↳ In Http request, we have various method / Parameter.
For Now we have 6 various properties.

HTTP req.
↳ URL (https://www.example.com/target)
↳ methods (GET, POST, PUT, DELETE, ...)
↳ headers (Content type: application/json)
↳ Query Params (optional)
↳ body
↳ timeout (60 second)

```
class HTTPreq {
    String url;
    "  method;
    map<string, string> header;
    "  queryParams;
    String body;
    int timeout;

Public:
    HTTPreq (url, method, header, ..., ...){
     this->url=url
       ;
    }
    void execute() {
        HTTP code
    }
}
```

```
main() {
  HTTPreq req =
  new HTTPreq (...)
  HTTPreq * req =
  new HTTPreq(url, method).
```

HTTPreq(url, m, h) {..}
HTTPreq(url, m, h, 2){}
HTTP() {..} //default.

- Consider, main() mein jo ki client ke pas hoga. Uss mein client ne sirf url, header pass kiya toh humein class mein new constructor banana pad rha hai.
- Aise hi bhot sare conductor ho jata hai.

Problem 2: Immutable object

- Problem is here of setters. We want that once our obj is created we shared should not be able to change them or ~~its value~~ i.e. obj should be immutable.
- We can't remove setters as it is impt. & it will create new problem.

Problem 3: Inconsistent state Problem.

- Instead of passing all parameters through constructors & we can pass only the important ones in the constructor & use setters for the remaining values & then run execute() method. In this way, we can solve constructor telescoping problem.
- We have to make sure now that execute() will only run if all methods are passed or set
- ex: We pass 3 obj as argument & set body only not timeout & queryparams then it shouldn't run.
- But if we do this it will not give us compile time error but runtime errors which is worst if we came to know about error at runtime.
- e.g. Even if we declare all variable but call execute() before some set methods still it will give us runtime errors.

```
class HTTPReq {
    ——
    ——
    ——
    ——
    ——
    ——
    HTTPReq (url, method, header) {
        this → url = url;
        this → method = method.
        this → header = header;
    }
    //Getter & Setter.
    execute() {
        }
}
```

```
main() {
HTTPReq * req =
new HTTPReq (url, m, h);
req → Setbody();
req → setTimeout();
req → execute();
}
```

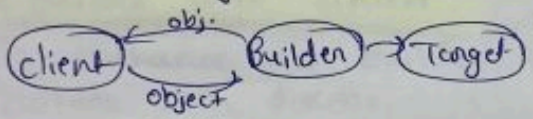
Problem 4: Validation

- Now let's assume I am responsible developer who always remembers to set all the required values & never leaves the obj. in inconsistent obj.
- Even then, I still want a way to validate that everything has been properly set before execute() is cld - especially when someone else is using my class.
- To ensure this, we are gonna add validation checks inside execute() to ensure required fields are present before proceeding.

```
void execute(){
    if (req -> getURL() == NULL)
        throw error or Lock;
    if (req -> getheader() == NULL)
        throw error or Lock;
}
```

- We have to perform this validation everywhere, we have used this req obj. This is known as scattered validation problem.

*** Solution of all problems :- Builder class**



↳ Whenever client wants obj. that time builder class will give obj to client from Target

* To understand how pattern implemented referred code.

→ Firstly, let understand about "this" keyword.

①
```
class A {
    int i1, i2;
    m1(){
        this;
    }
}
```
→ this keyword return kar rha hai. A ka current reference kiska return rha hai ⟹ client ko jisne A ka obj banaya hoga.

②
```
class A {
    int x1, x2
    A(){
        this.x1 = x1
        this.x2 = x2
    }
}
```
this matlab current value ko reference of in values ko set kardo.

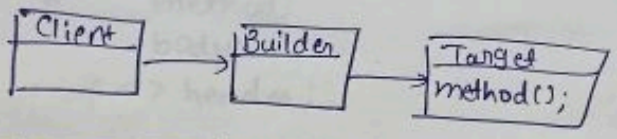
* Analyzing dia. of Builder Pattern.

To build a obj step-by-step

Request()
→ .withURL(..)
• withmethod(...)
:
• build() ⟶ Terminating method as it terminates the chaining of build.

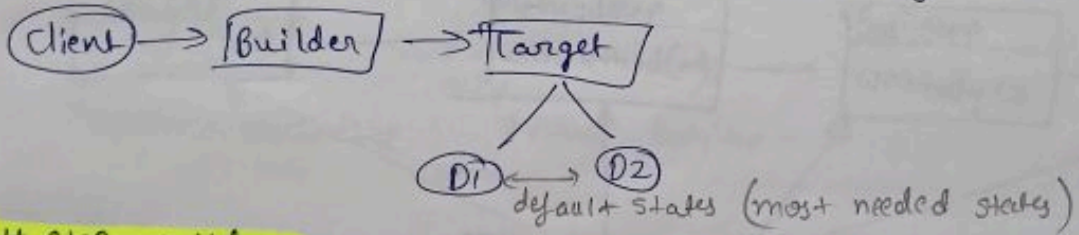
- Other method known as Intermediate method as they do chaining.
- All intermediate method returns obj of builder then our terminating method atleast provide us request obj by performing validation.
- Efficient than setters due to ⇒ Provide immutability, improve readiability.
- Main task of builder is to provide is a req. obj. slowly first made are then method & so on. It will not stop until we call final build() method.

#UML dia of Simple Builder.



Builder with Director.

- To enchance our builder functionalities or its power, we introduce builder with director.
- It provides reusable builds means it stores the pre-existing default states as method whenever anyone ask for it then give it to them.
- Whenever you create any obj. it has some default states.

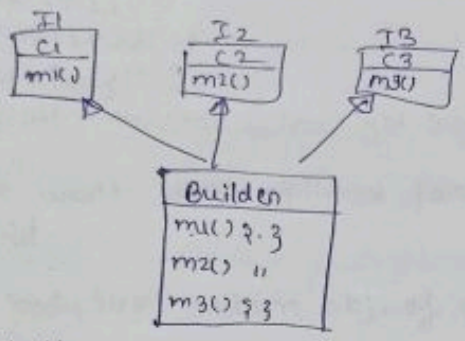


Step Builder

- Let's take example of Pizza
Whenever we buy pizza they ask some questions like → crust, Sauce, Topping, cheese.
- Some obj. are needed to be created in an specific order.
- This order Maintainability is present provided by step Builder.

(*) Two key points
- create obj step-by-step.
- If required, validate whether you declare all parameters or not (optimal).

- Before going further, let us understand about multiple inheritance.



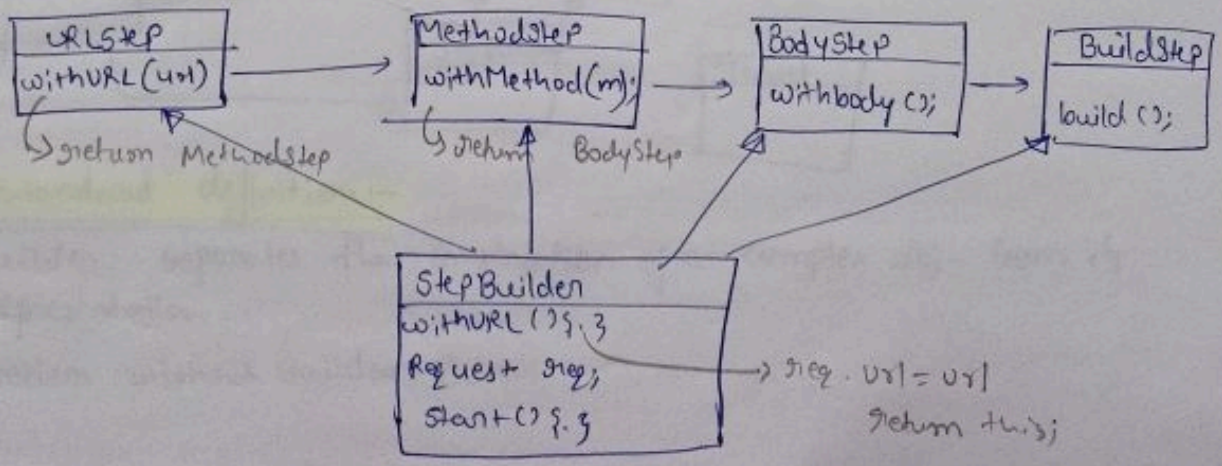
- Only pattern to use multiple inheritance.

- Use tradeoff → more strictness ⇒ use mutiple inheritance.

Working of step Builder

```
- class Request {
      String url;
      "  method;
      "  body;
      map<> header;
  }
```

- Ab humne builder class banaya. Till now, jab bhi hum withURL then it return a obj. of builder class which also us to access any of the method in any order.
- But for step-by-step execution that made a separate pure abstract class of every parameter.
- To resolve it first class should return obj. of next abstract class.



① We gave a reference of stepBuilder class to client.

② It says call start() to start obj creation.

③ Client call start() ⟹ start() return URLStep object.
Client get URLStep object.

- withURL(..)
- withMethod(..)
- withBody(..)
- Build() ⟹ Stop (client get request reference).

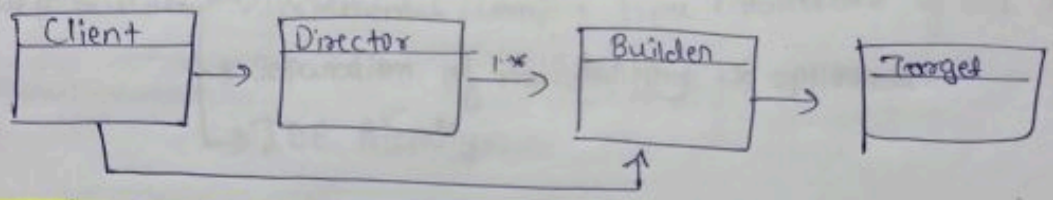
- If we want some optional parameter we can make optionalStep inplace of build.

Now bodyStep return obj. of optionalStep

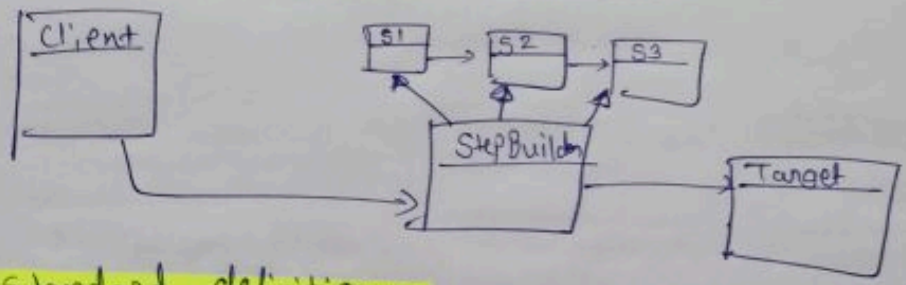
OptionalStep
withHeader();
withTimeout();
withBuild();

- After body, when it return optional step obj., we may or may not call its method.
  - withHeader(→) :→ return its own obj.
  - withBuild(..) :→ returns req. obj.

#Standard UML for Builder with Director:



#Standard UML for step Builder



#Standard definition:-

Builder separates the construction of a complex obj. from its representation.

#Problem without Builder Pattern

Summary

#Problem without Builder Pattern.

① Constructor Explosion
- Every new optional param requires a new constructor overload.
- Calls become unreadable as you pass empty/dummy values for skipped fields.

② Inconsistent obj. status.
- Partially built obj. may be used before all required data is set.

③ Mutable obj.
- Exposing setters means client can change the obj. any time.

Normal Builder Pattern
- clear, readable obj. construction.
- Single centralized validation.
- Immutable obj.
- No constructor overload.

Director Builder → Resuable Builds.

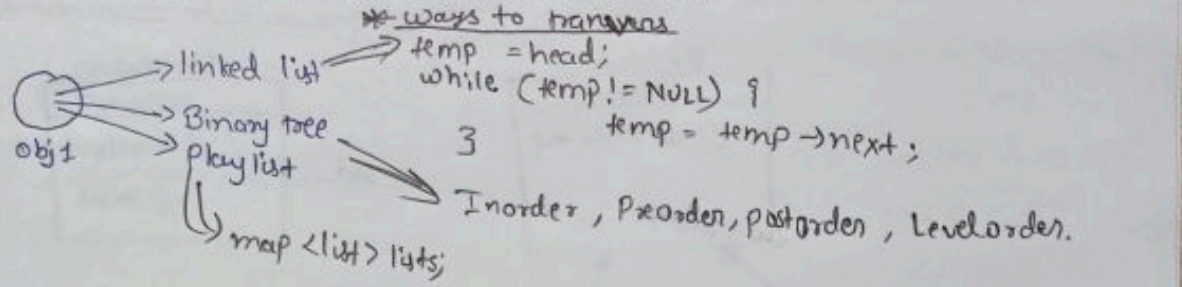
Step Builder → Incremental / compile time enforcement of all required field.
- Separation of mandatory vs optional.
- IDE friendly.

Iterator Pattern

* Introduction

↳ It is used by u in every data st.

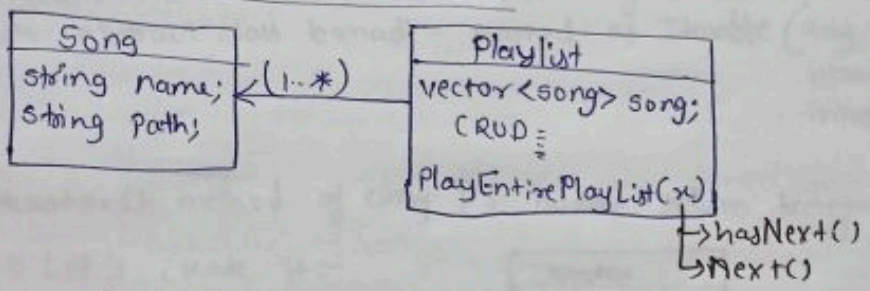
↳ Suppose, we have a obj. & we have to traverse it:- Traversing of that obj. will depend on data st. used to store it.



- Along with all these we have another way of traversing kin as "iterators".

↳ list.get.Iterator => it
↳ hasNext()
↳ next()

* Why do we need Iterators?

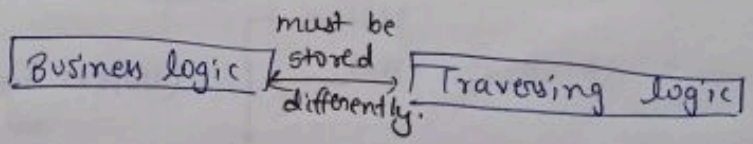


- Initially, hum vector mein store kar rhe the songs ko, we can iterate using loop :- for (song : song)
  song->path;

- Later, we want to store in L.L. so we need to change the traversing logic as it is different for L.L.

- Which breaks SRP Principle, as we are storing both business logic & traversing logic in some class.

- Therefore,

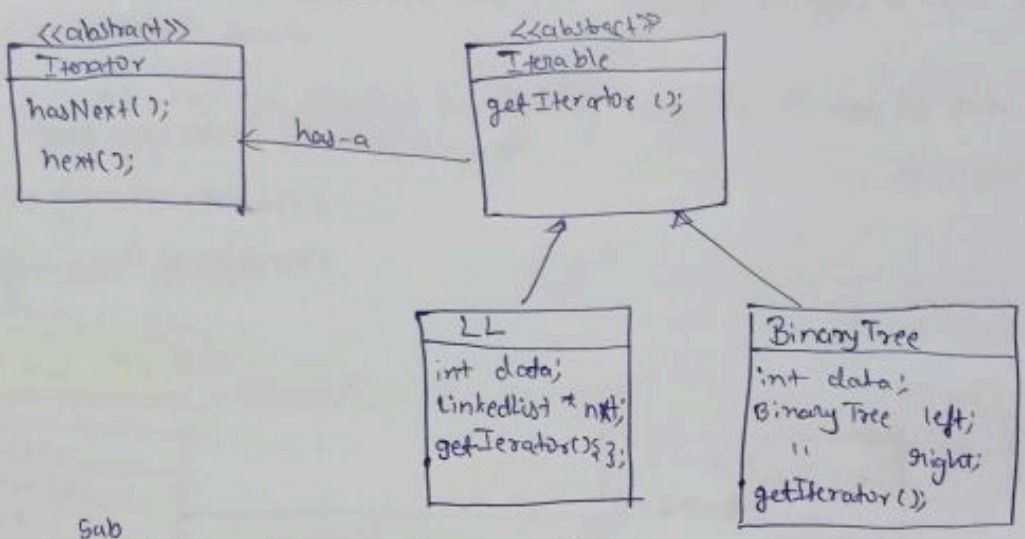


- Which means playlist should not know how to traverse, it just call one method & get another way.

↳ we will use Iterator for that => tho. more work, but to prevent breaking SOLID Principle.

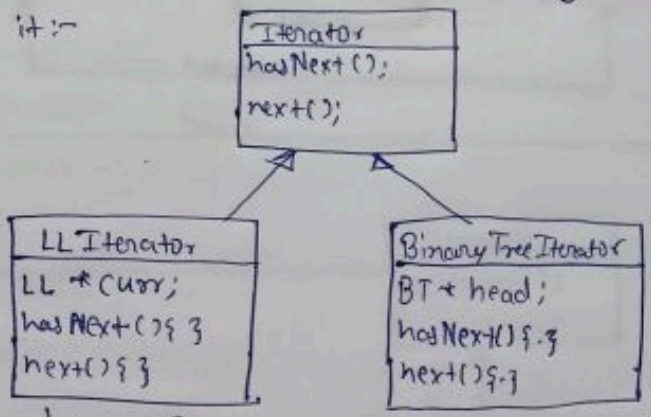
- Using Iterator we will be able to follows SRP even if we change it's ds.
- Method par koi effect hi nhi, chahiye, kyuki hum change ~~it's~~ nhi, karege as they will just call hasNext() & next() method of iterator passed in them.

* UML dia for Playlist example



- Ab hum ne sub DS mein baar-baar getIterator() method banane ke jagah abstract class banadi - named as Iterable (any ds which can be iterable should be inherit it).

- getIterator() method of every DS. humme return karega ek iterator
  → Let's create it:-



* How LL. traversing happens?

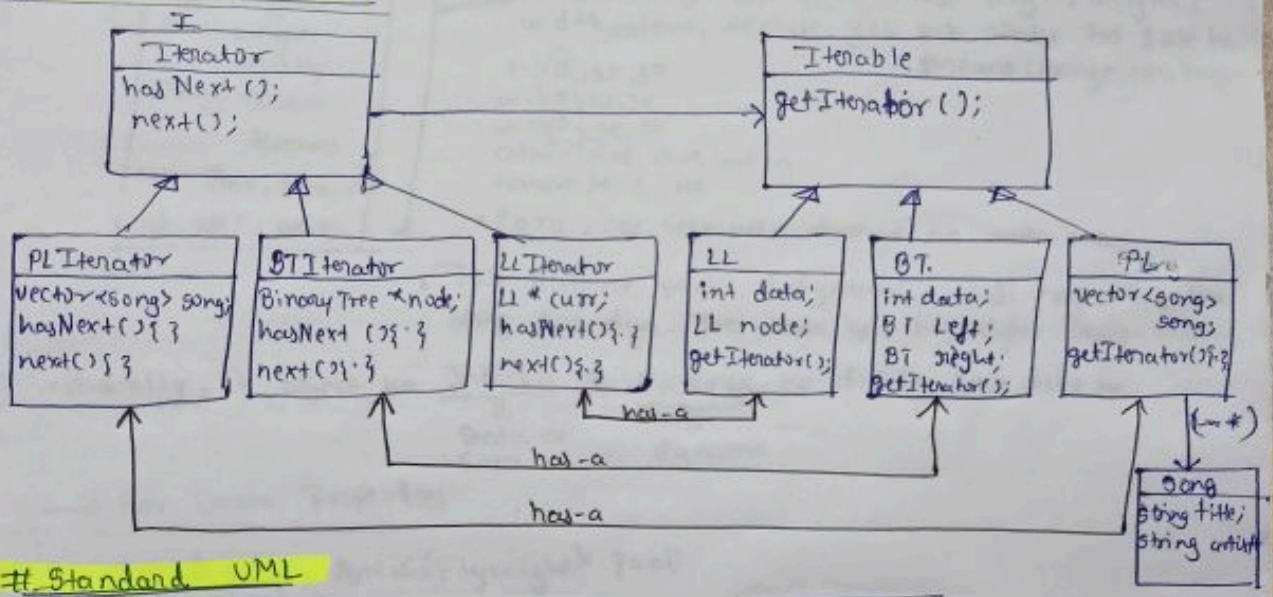
```
hasNext(){
    if(curr->next != NULL)
        return true;
    else
        return false;
}

next(){
    curr = curr->next;
}
```

- LL "has-a" LinkedList Iterator.
- Binary Tree "has-a" BinaryTree Iterator.
- Now, Let's assume playlist has songs & it also has its own iterator.

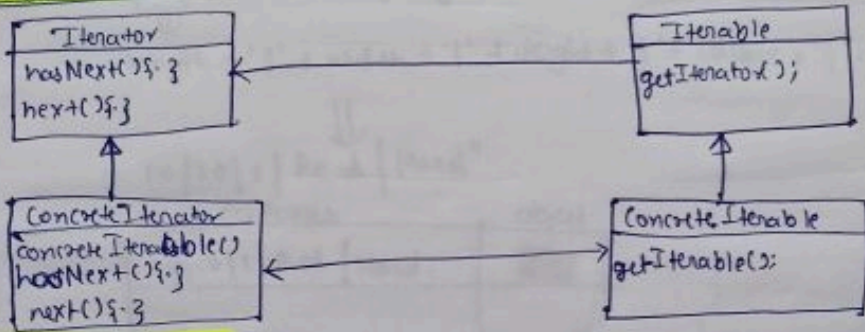
* How Playlist+Iterator will work?
  - Client call karega getSongByName() method ko in Playlist
  - Playlist calls its getIterator() method jo traverse karega, & uske baad it gets it's playlist iterator.
  - Fir woh traverse karenga playlist ke list of song ko & use hi pata hoga ki list stored in vector / linkedlist, etc.

```
while (it -> hasNext() {
        it -> next()
}
```

UML Final Dia.



Standard UML



Standard Definition

↳ Iterator provides us a way to access the elements of an aggregate obj; sequential without exposing its underlying architecture.

Flyweight Design pattern

* Introduction
  - Solve specific-use case. It is used to save RAM.
  - Jab humein RAM ki space bachani ho toh Flyweight Design ko use karte hai.

* How it can help?
  - Flyweight ko wahi par use kar skte hai jaha par multiple object ban rhe ho (like million/billion mein).
  - Consider e.g. Ek game hai jismein, gun hai aur shot kar rha hai multiple asteroid ko.

• Ab multiple asteroid hai multiple object banenge.

```
AsteroidFlyweight
int length;
    weight;
    width;
String colour;
       texture;
int Posx, Posy;
int velx, vely;
```

→ Yeh saare obj. hai (Properties) length, weight, width, colour, texture yeh sab static ho skte hai means change nhi hoga.

l = 10, 20, 30
w = 10, 20, 30
w = 10, 20, 30
colour = Red, Blue, Green
texture: H, S, HS

• Posx, Posy, velx, vely change ho baar-baar.

• Toh humne static & dynamic wali properties ko alag kar diya. Yehi hota hai Flyweight Design pattern.

• Basically, Ek object ko Intrinsic (⇓ Static or Same) & Extrinsic (⇓ dynamic) ko divide kar dete hai.

→ For same Properties.

```
map<key, AsteroidFlyweight> pool;
```

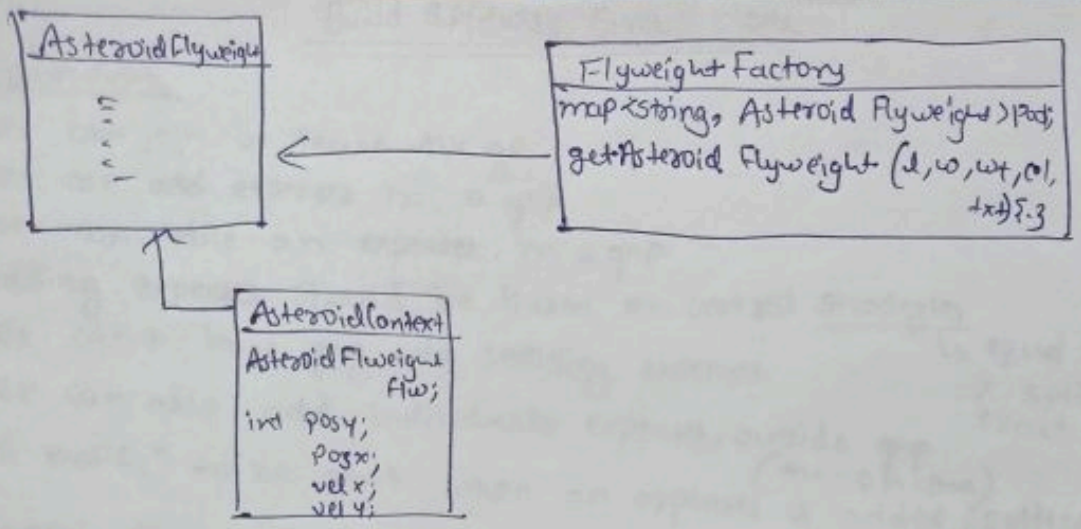
key ⇓

```
length + '|' + width + '|' + weight + '|' + colour + '|' + texture;
```

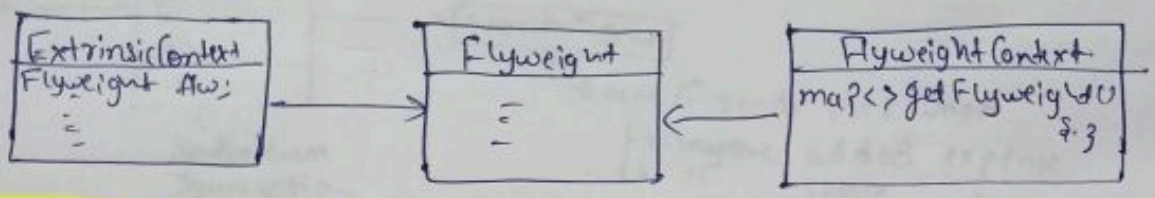
⇓⇓

"10|10|1|Red|Hard"

Properties	object.
10|10|1|Red|Hard.	



#UML - Standard



#Standard Definition

- Flyweight uses sharing to support large no. of fine grained objects efficiently.
- Intrinsic state: (Stored among objects)
- Extrinsic state: (Supplied by client externally).

#Real World Use-case

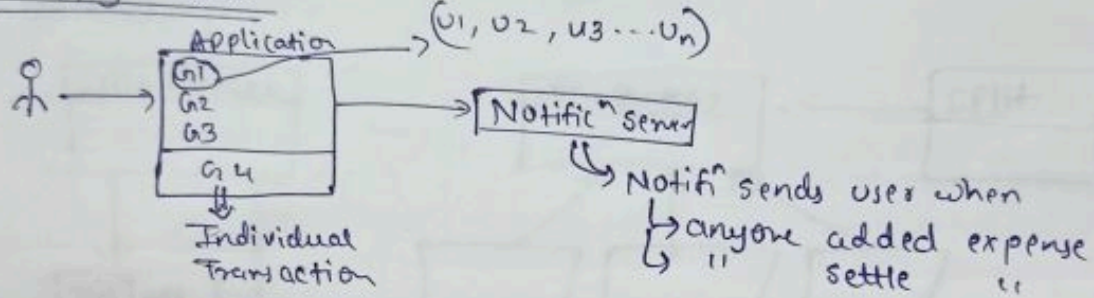
1. Game GTA-5
2. Text editor

Build Splitwise Group clone

* Requirements

- User can join or leave the gp.
- User can add expenses in a grp.
- User can settle an expenses in a grp.
- Adding expenses should be based on several strategies
  - equal split,
  - % split,
  - exact split.
- User can't leave grp w/o settleting expenses.
- User can also add individuals expenses, outside grp. (one-on-one)
- A notific^n to be sent when an expenses is added / settled.

* Happy flow



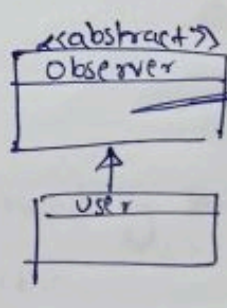
Note:- To Simplify Transaction
- implement using Greedy Algorithm
- Add on features to reduce no. of transactions overall in grp.

* Process

- Notification Server (observer pattern).
  - We will use id to add in grp, split expense based on id.
  - ∴ In map<string, double> expense,
    - string ⇓ id.
    - double ⇓ amt
  - How values is stored in map

id . . .
u1 → Paid → 800
- u2 ⇒ 200
- u3 ⇒ 200
- u4 ⇒ 200

} u1 ko denge.

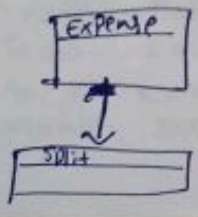


Individual / Grp. dono mein map banega like u1. Ise liye

u2 : 200
⇓ key ⇓ value.

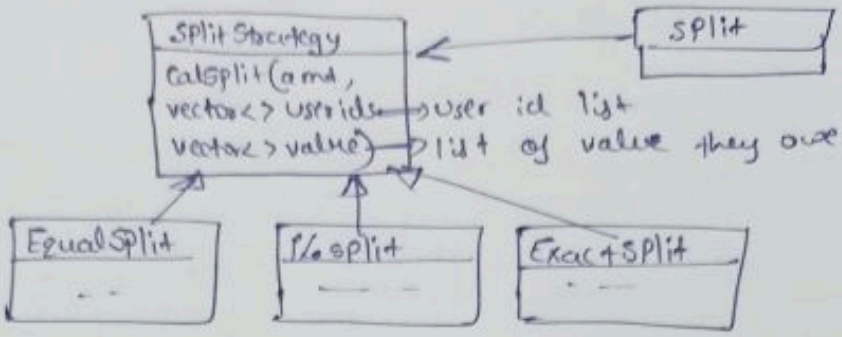
u2	200
u3	200
u3	200

- Update Bal() ⇒ will update value in our balance map.

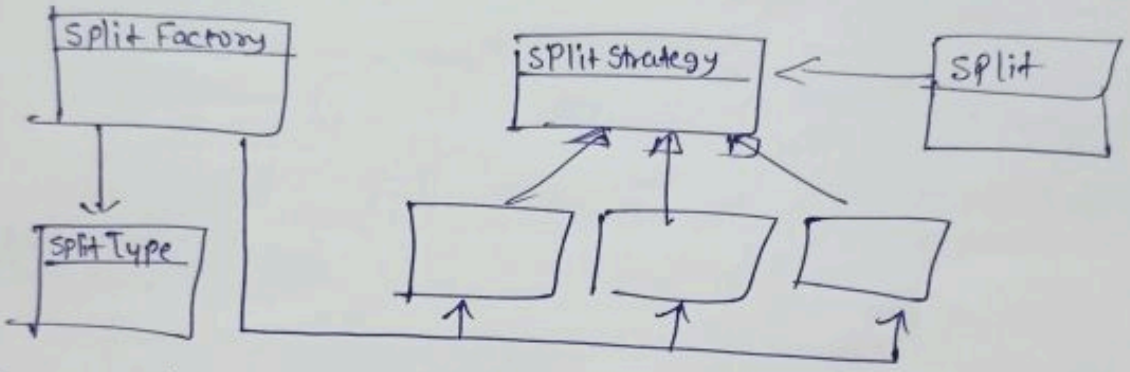


- Hum ise store karne ke liye: vector<pair<uid, amt>>
- But calling it as second, again passing & storing will be quite complicated
- ∴ We will create split Class.

- Now we need different strategies to split money among all the members & we will use strategy pattern so that we can add more strategy in future.



- To create this SplitStrategy we will need to make SplitFactory.



- Group class
  ↳ observable class hogi jis ko create karne ke liye 2 methods
  ① different class for different function like add, remove.
  ② All in One.

We will choose method ②.

∴ It is responsible for observing & handling business logic too.

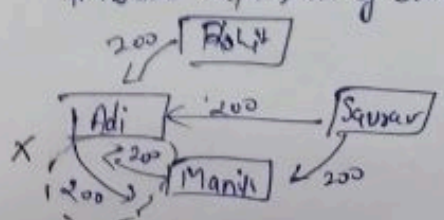
*Greedy Algorithm : simplify Transaction

↳ DebtSimplifier

① Aditya paid ₹800 for lunch (Equal)
Rohit : 200
Saurav : 200
Manish : 200

② Manish Paid ₹700 for Dinner (Exact)
Saurav: 200
Aditya : 200
Manish: 300

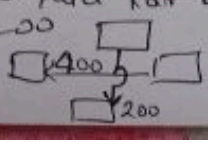
*Visual represent of data



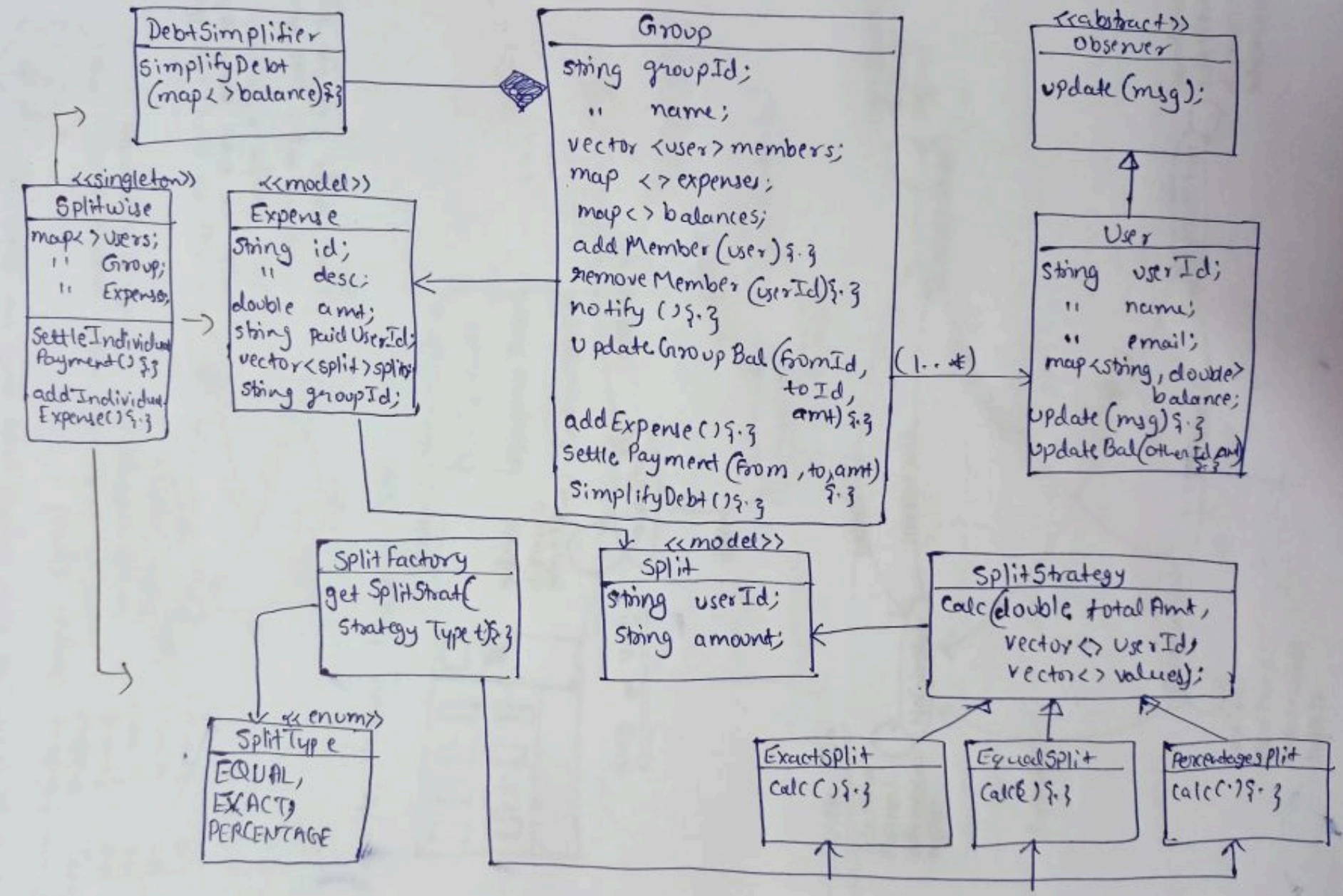
Transac^n 1:- T1, T2, T3

Hum Saurav se Adi ko 400 paid kar denge
And Rohit se Manish ko 200

∴ Transac^n 2 : T1, T2.



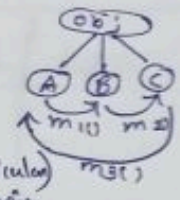
UML dia


DebtSimplifier
SimplifyDebt (map<> balance){}
Group
string groupId;
" name;
vector<user> members;
map<> expenses;
map<> balances;
addMember (user){}
removeMember (userId){}
notify (){}
UpdateGroupBal (fromId, toId, amt){}
addExpense (){}
SettlePayment (from, to, amt){}
SimplifyDebt (){}
<<abstract>>
Observer
update (msg);
<<singleton>>
Splitwise
map<> users;
" Group;
" Expense;
SettleIndividualPayment(){}
addIndividualExpense(){}
<<model>>
Expense
String id;
" desc;
double amt;
string paidUserId;
vector<split> splits;
String groupId;
(1..*)
User
String userId;
" name;
" email;
map<string, double> balance;
update (msg){}
updateBal(otherId, amt)
SplitFactory
getSplitStrat(
StrategyType t){}
<<model>>
Split
String userId;
String amount;
SplitStrategy
Calc(double totalAmt,
vector<> UserId,
vector<> values);
<<enum>>
SplitType
EQUAL,
EXACT,
PERCENTAGE
ExactSplit
calc(){}
EqualSplit
calc(){}
PercentageSplit
calc(){}

State Design Pattern

* Introduction:-

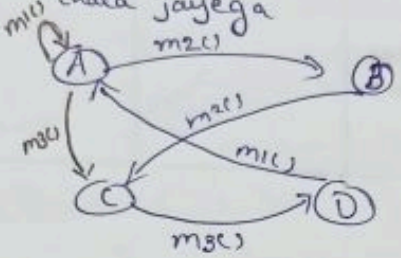
↳ Ek object hai jo kuch limited state mein rhe skta hai



⊙ State Machine Diagram (SMD)

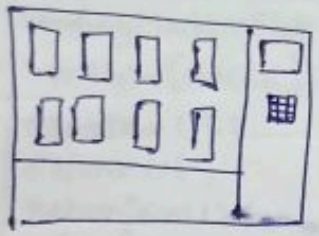
Obj	Method
A	m1()
B	m2()
C	m3()
D	m4()

- Koi bhi state mein koi ek method (particular) ko call karenge toh woh dusre state mein chala jayega



- A obj ne m1() call par state nhi kiya hai isliye loop design kiya hai.
- A obj ne m3() call par state change kar liya by ©.

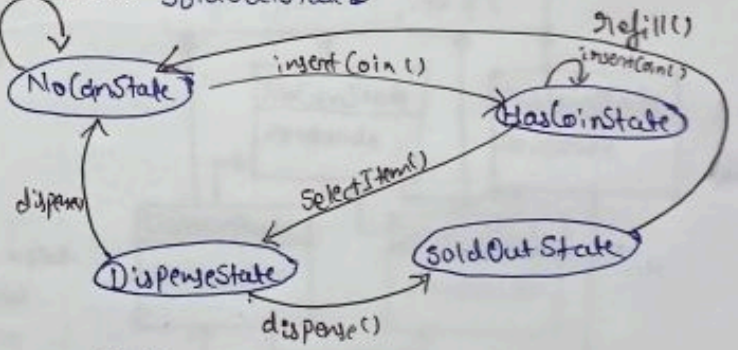
Example:- Vending Machine. (VM)



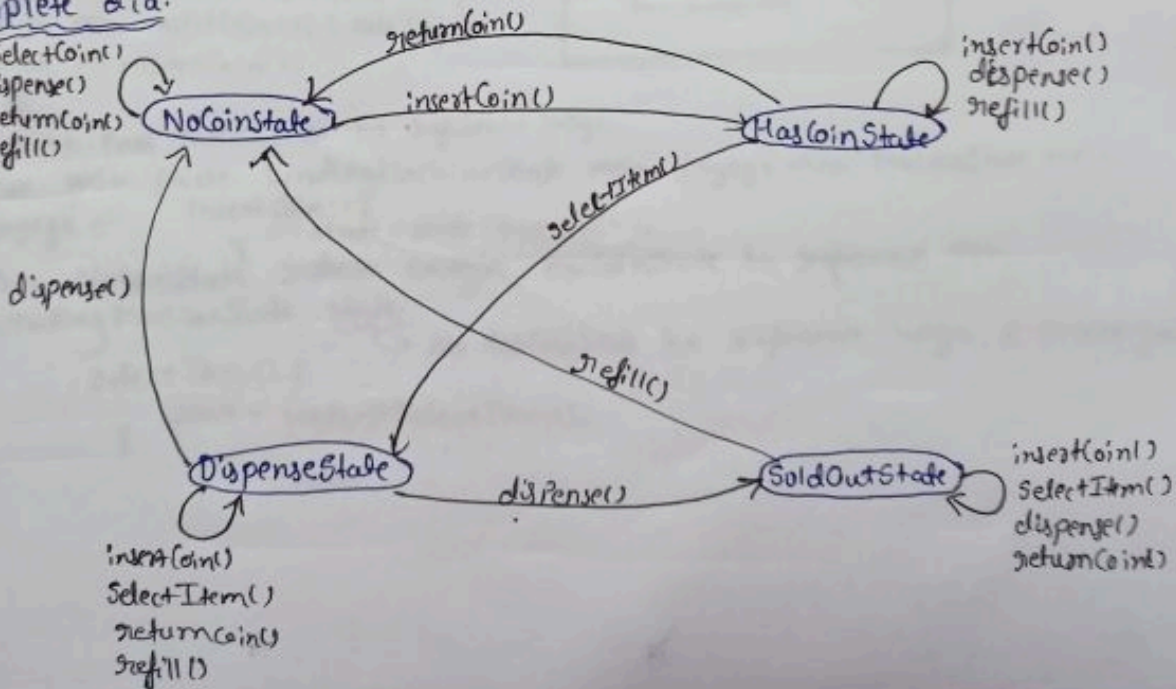
States ⇒ NoCoinState, HasCoinState, DispenseState, SoldOutState

refill()
dispense()
selectItem()

SMD of VM:-



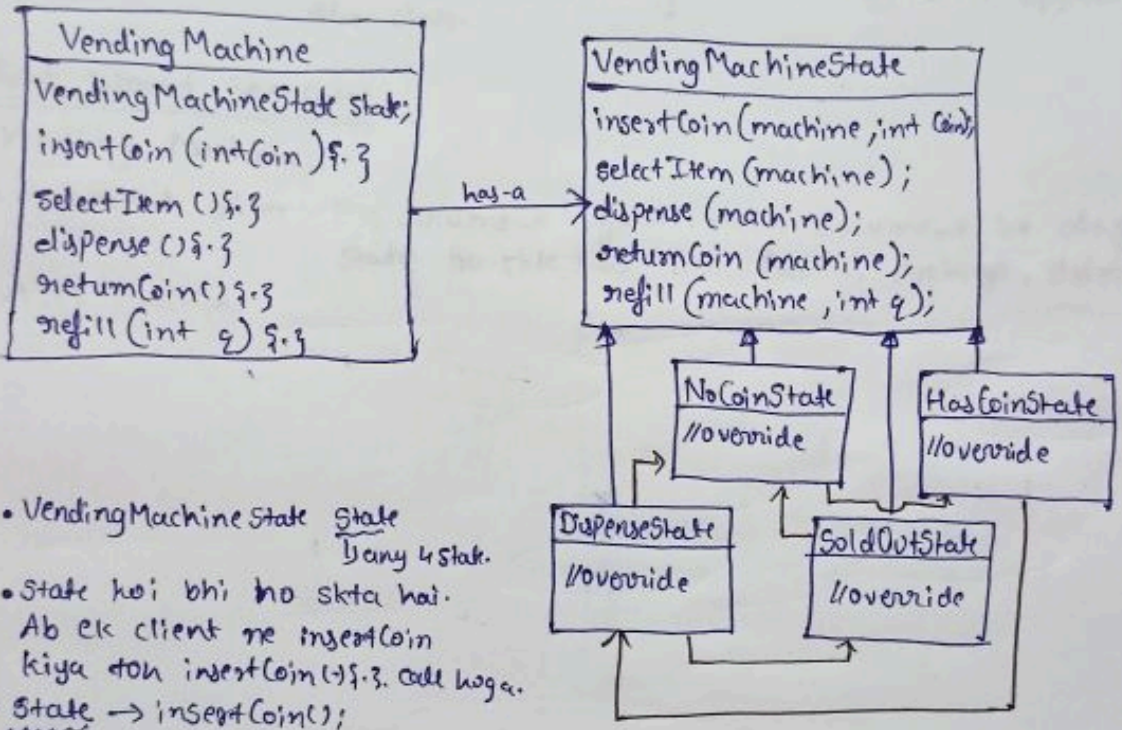
* Complete dia.



* Table

Action / States	InsertCoin	SelectItem	Dispense	ReturnCoin	Refill
NoCoinState	HasCoinState	—	—	—	—
HasCoinState	—	DispenseState	—	NoCoinState	—
DispenseState	—	—	NoCoinState SoldOutState	—	—
SoldOutState	—	—	—	—	NoCoinState

* UML dia



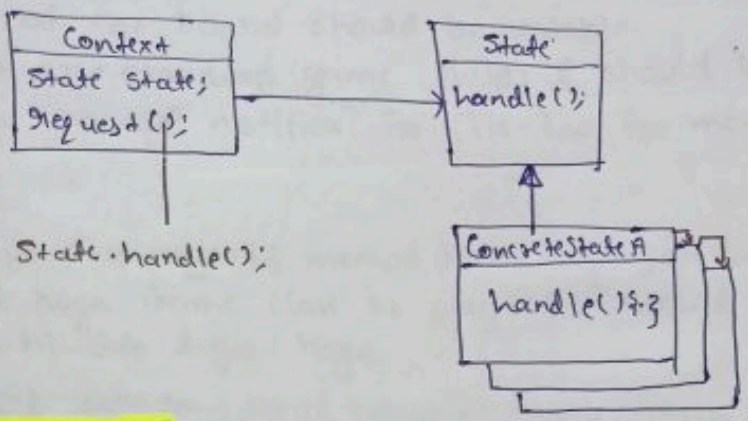
- VendingMachineState State
  ↓ any 4 state.
- State koi bhi ho skta hai.
  Ab ek client ne insertCoin kiya toh insertCoin() {..} call hoga.
- State → insertCoin();
  ⇓
  Iss ke pass NoCoinState ka reference hoga.
  jise woh phele VendingMachineState mein jayega then NoCoinState mein jayega.

```
insertCoin() {
    state = state -> insertCoin();
}
```

- Phir NoCoinState return karega HasCoinState ka reference toh VendingMachineState state.
  ↳ Ab HasCoinState ka reference hoga & process goes on

```
selectItem() {
    state = state -> SelectItem();
}
```

* Standard UML dia



* Standard Def:- It allows an obj to alter its behaviour when its internal state changes. The obj will appear to change the class.

* Real World Use-Case:-

① Vending Machine

② Document editor:- Ek document system mein document ke alag-alag state ho skte hai like Publish, Archive, delete, etc.

③ ATM Machine

Build Tic Tac Toe Game

* Requirements

- Size of the board should be scalable.
- There are standard game rules & should be further extendible.
- Allow an app notifications for Tic-Tac-Toe moves, wins, draws, etc.

* Work

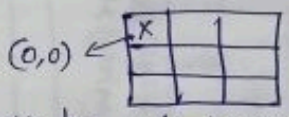
- Client ke pass play() {;} method hota hoga jo woh call karegi. Jo ki directed hoga Game class ke play() {;} method ko. play() {;} iss ke andar hi sab logic hoga.
- play() {;} calls to → deque <Player> players; (Player ko choose karega/nikalega deque mein se).

↓

Fir player class mein hum symbol check karege Player ka.

↓

consider, player ne game khela aur symbol ko place kiya.



↓

Ab play() {;} Phir se → Rule class ko call karega. Jis mein woh isValidMove(board, r, c); ko bolega check karne ke liye.

↓

Ab isValidMove(...), StandardRules class ke pass jayega aur isValidMove(...) {;} ko call karega. Jis mein hum Board b ka reference Pass kar rhe hai.

↓

Ab Board class mein isCellEmpty(..) {;} ko call karega aur easily check karega.

↓

Ab valid hone ke baad Fir se play() {;} method Rules class mein checkWin(.) ko call karega.

↓

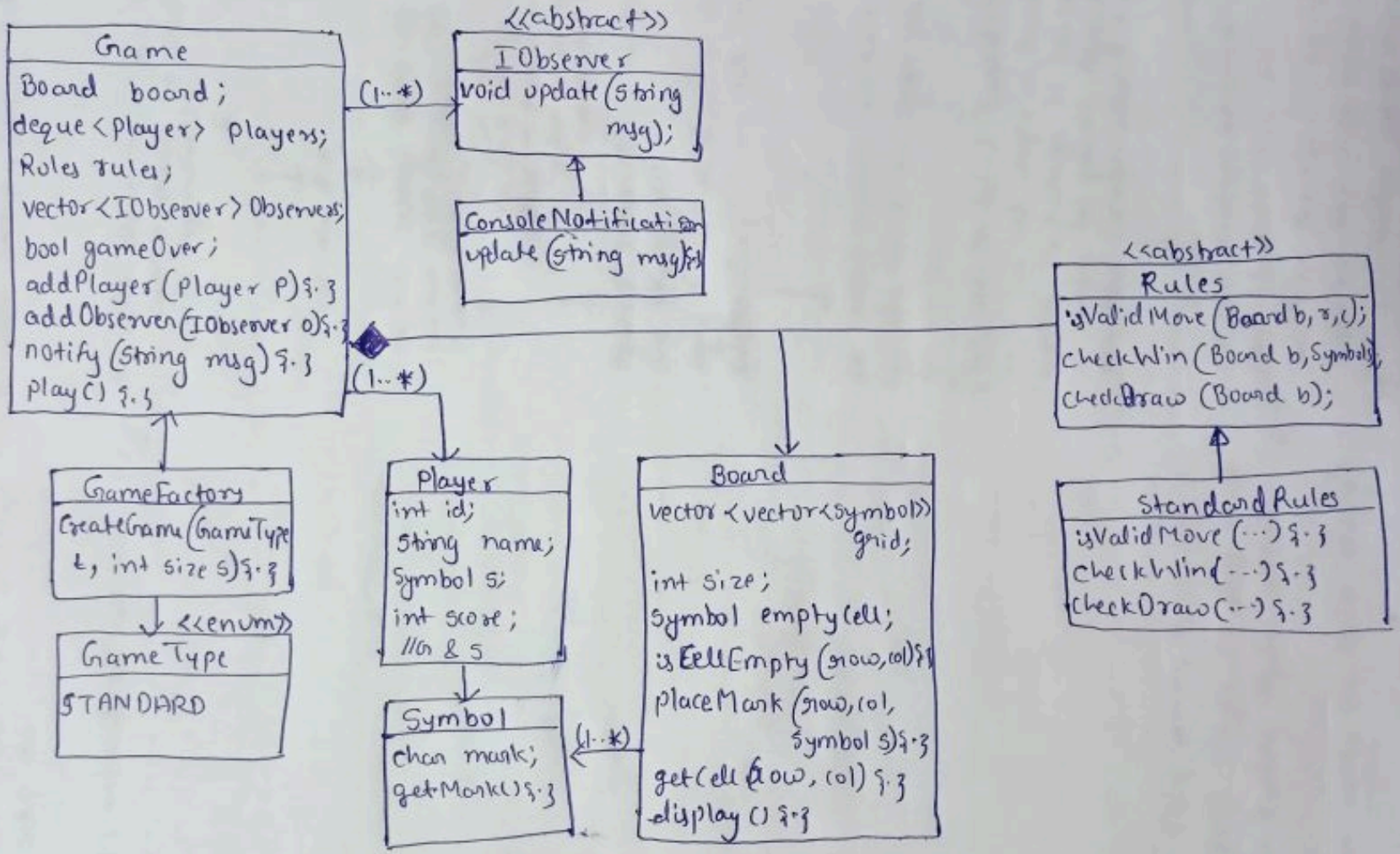
Agar F return hua toh play() {;} Phir se Rules class mein jaa ke checkDraw(..) ko call karega aur check karega ki match draw toh nhi hogya

Saare cell mein jaa kar check karna ki full toh nhi hogya hai. Agar F return hua toh play() {;} Fir se deque mein jaayega

checkWin(.) board ke symbol ko dekhega here it is "X" aur check karega X ke horizontal, vertical & diagonal side. And T/F return karega.

→ Process go on. (bich-bich mein notify bhi call karega.)

UML Dia


Game
Board board;
deque<Player> players;
Rules rules;
vector<IObserver> Observers;
bool gameOver;
addPlayer(Player p)
addObserver(IObserver o)
notify(String msg)
play()
(1..*)
<<abstract>>
IObserver
void update(String msg);
ConsoleNotification
update(String msg)
<<abstract>>
Rules
isValidMove(Board b, r, c);
checkWin(Board b, Symbol
checkDraw(Board b);
GameFactory
CreateGame(GameType t, int size s)
<<enum>>
GameType
STANDARD
Player
int id;
String name;
Symbol s;
int score;
//s & s
Board
vector<vector<symbol>> grid;
int size;
Symbol emptyCell;
isCellEmpty(row,col)
placeMark(row,col, Symbol s)
get(cell&row, col)
display()
Symbol
char mark;
getMark()
(1..*)
StandardRules
isValidMove(...)
checkWin(...)
checkDraw(...)

Builder Snake & Ladder Game

* Requirements

- Size of the board should be scalable.
- There are standard game rules & should be further extendible.
- There can be game setup strategy like Random setup, custom setup, standard setup etc.

* Work

- Client play() ko call karta hoga → then → woh deque mein jaa kar current player ko nikalega.

while(!isgameOver) {

}

↓

roll the dice

↓

Ab hum bolenge Rules class mein jaa kar isValidMove(..). Yeh T/F return karega:

↓

Yeh check karega - Consider, Player 99 position par hai. Aur 6 dice par no. aaya hai. Toh move nhi kar payega

Agar F return kiya hai toh dice phir se roll hoga.

Agar T return kiya hai toh (Player) calcNewPos(....) ko call karega.

↓

Ab current position par check kar rha hai ki koi snake yaa ladder toh nhi hai.

{Yeh calcNewPos(..) Board class mein jaa kar map<int, BoardEntity> entities; ko puchega ki koi snake or ladder hai yaa nhi. Agar hai toh new position allocate kar do}

↓

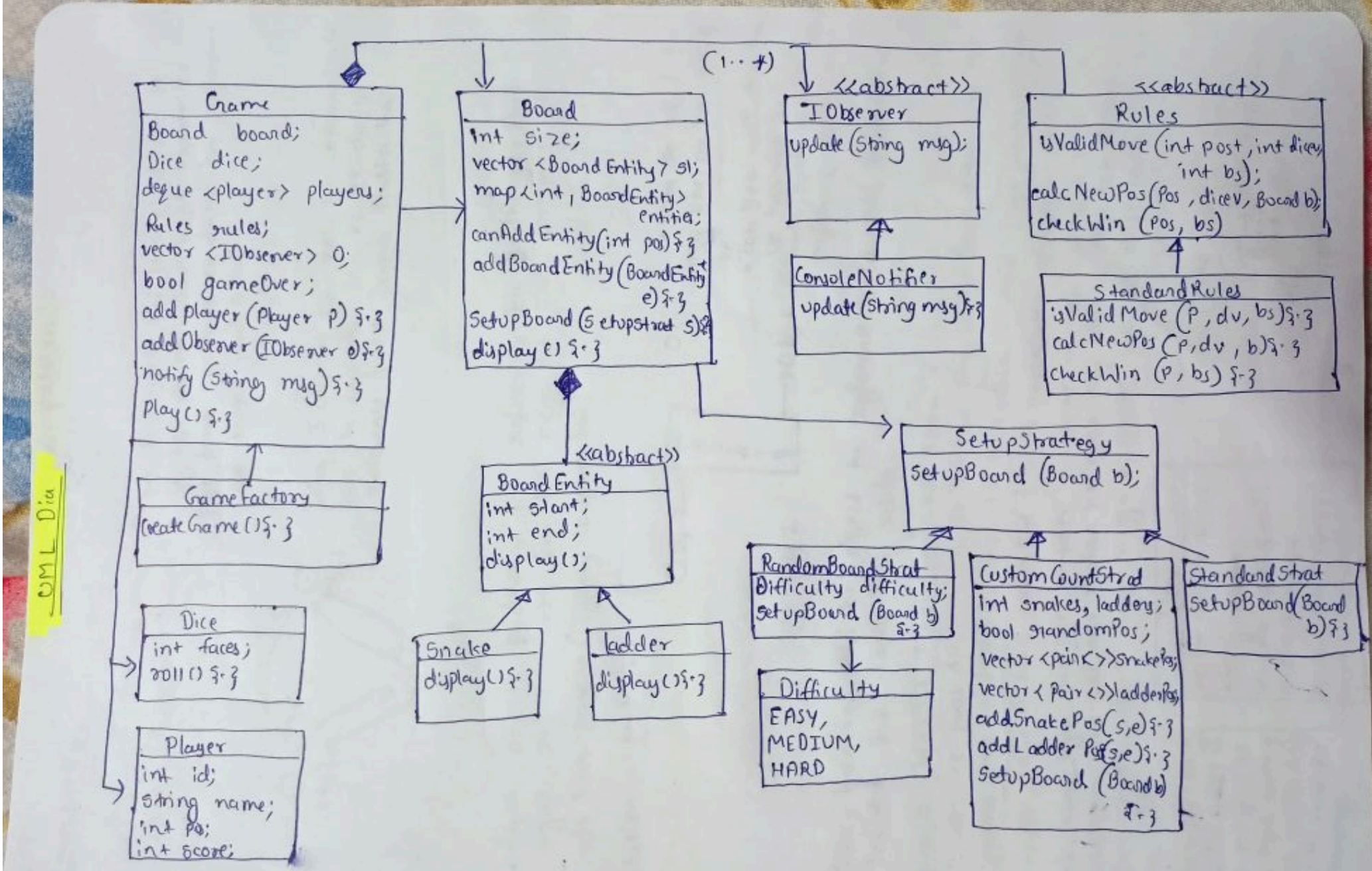
Ab new position ke baad Play(..) ko Rule class mein jaa kar checkWin(..) ko call karta hai aur check karta hai ki new position end of board ko position toh nhi hai. Agar hai toh winner ho gya.

←

Agar checkWin False return karta hai toh Play() ko next player ko play karne ko bolega from having deque current player.

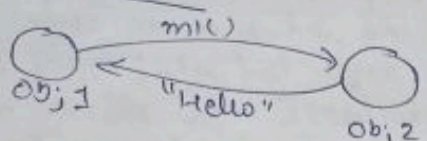
(Bich-Bich mein notify bhi karte rhenge)

UML Dia

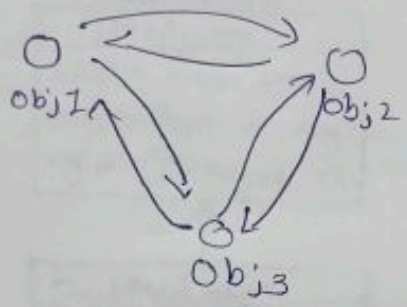

(1..*)
Game
Board board;
Dice dice;
deque <player> players;
Rules rules;
vector <IObserver> O;
bool gameOver;
add player (Player p) {..}
add Observer (IObserver o) {..}
notify (string msg) {..}
play() {..}
Board
int size;
vector <BoardEntity> sl;
map <int, BoardEntity> entities;
canAddEntity(int pos) {..}
addBoardEntity(BoardEntity e) {..}
SetupBoard (SetupStrat s) {..}
display() {..}
<<abstract>>
IObserver
update (string msg);
ConsoleNotifier
update (string msg) {..}
<<abstract>>
Rules
isValidMove (int post, int dicev, int bs);
calcNewPos (Pos, dicev, Board b);
checkWin (pos, bs)
StandardRules
isValid Move (P, dv, bs) {..}
calcNewPos (P, dv, b) {..}
checkWin (P, bs) {..}
GameFactory
createGame() {..}
<<abstract>>
BoardEntity
int start;
int end;
display();
SetupStrategy
setupBoard (Board b);
Dice
int faces;
roll() {..}
Player
int id;
string name;
int Po;
int score;
Snake
display() {..}
ladder
display() {..}
RandomBoardStrat
Difficulty difficulty;
setupBoard (Board b) {..}
Difficulty
EASY,
MEDIUM,
HARD
CustomCountStrat
int snakes, ladders;
bool randomPos;
vector <pair<>>snakePos;
vector <pair<>>ladderPos;
addSnakePos(s,e) {..}
addLadderPos(s,e) {..}
SetupBoard (Board b) {..}
StandardStrat
SetupBoard(Board b) {..}

Mediator Design pattern

Introduction



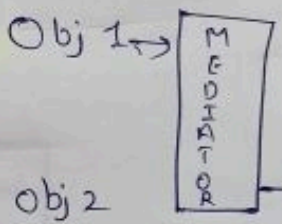
- Yeha 2 objects hai joh ek-dusre se baat kar rhe hai. Aur baat karne ke liye dono ke paas ek-dusre ka reference hold karna padta hai.



Yeha 3 objects hai joh ki communicate kar rhe hai. Aur inn ko ek-dusre ka reference hold karna padta hai.

- Agar objects badhenge toh references bhi badhenge. Aur agar reference nhi hai kis mein toh woh communicate nhi kar payega (Tightly coupled hai).

Solution :- Mediator



- Yeha pe obj 1 ko msg bhejna hai obj 4 ko.
- Toh yeh mediator ka use kar rhe hai jis mein sare objects ka reference hai.

- Yeha humein kisi bhi object ka reference hold nhi karna hai Mediator hold karta hai sabka.

Example : Without Mediator "Chat Room".

- Ek user ke paas dusre users ki list hoti [User]s issliye loop draw kiya hai.
- Humare pass sendAll(--), send(--), recieve(--) ka method hoga joh ki common hai.
- Consider, aage features badh gye that is mute(user user);
- Toh hum kya karenge, consider Rohan ne mohan ko mute kiya toh Rohan ek user hai aur mohan ko agar msg bhejna hoga toh mohan apna (khud ka) naam check karega ki dusre user ne usse mute kiya hai yaa nhi.

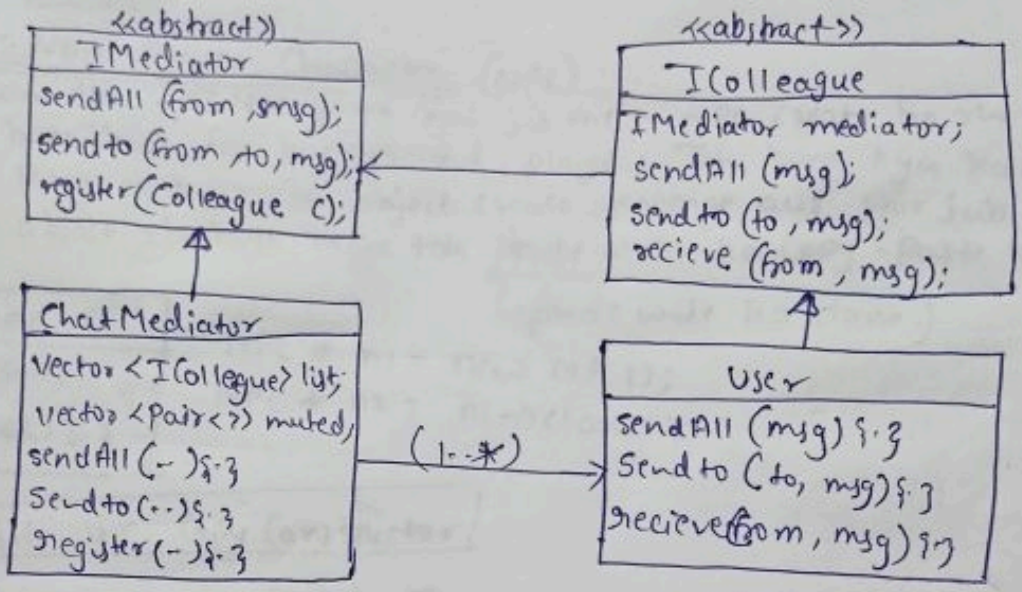
```
User
vector<user> users;
string name;
sendAll(msg);
send(msg, to);
recieve(msg, name);
mute(user user);
```

vector<user> mute; (Yeh list of user ko store karega jise hum mute karenge Rohan ka list dikhega waise hi multiple list banenge.

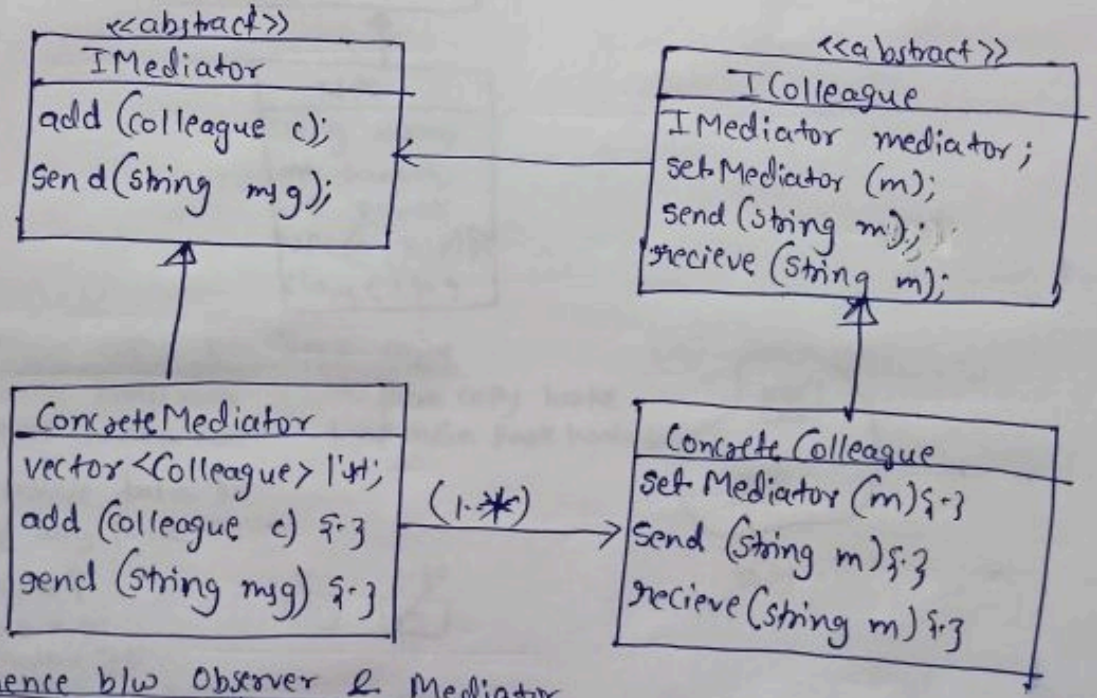
```
for(auto user : users){
    if(user->getMethod(name))   // Rohan, Mohan
        continue;
    else
        user->recieve();
}
```

• Example: with Mediator "Chat room".

• Colleague ko baki colleagues ki list maintain nhi karna padta hai. colleague mein se koi bhi method call hota hai toh woh Imediator ke paas jata hai aur woh particular method call ho jata hai for specific user. IMediator ke paas khud ki list hai jis mein sare users hai.



Standard UML Dia



Difference b/w Observer & Mediator

- Observer: jab bhi state change ho sab ko pata lagye. like Notify Server.
- Mediator: 2 colleagues ko aapsmein interact karvana. like in chat room / match making system.

Standard Definition

↳ Defines an object that encapsulates how a set of objects interacts, & promote loose coupling by prevending them from referring in each other.

Real-World Use-case

1. ChatRoom
2. Online Match makers (Chess, ...).

Prototype Design Pattern

* Introduction

- Consider class A jis ke humko multiple object chahiye → A* a1 = new A();
  → A* a2 = new A();
  → A* a3 = new A();
- Baar-Baar object create karna ek expensive opern hai.
  ↳ DB connection, complex calculat^n, etc.

Solution: Prototype

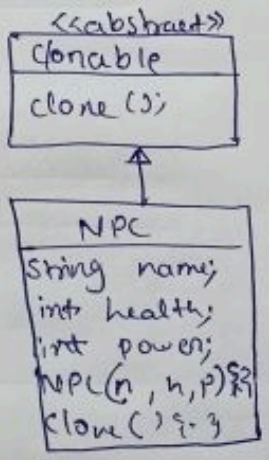
Example:- Non-Player Character (NPC).
Consider ~~free~~ fire game hai jis mein NPC create ho rhe hai baar-baar for background player. Toh hum kya karenge, Hum ek baar hi object create karenge aur phir jab new object chahiye hoga toh ~~pehle~~ pehle wale ko copy-paste kar denge.
↳ (pehle wale ka clone.)

NPC
//expensive opn
clone() {-}

```
NPC * n1 = new NPC();
NPC * n2 = n1->clone();
```

We will use Copy Constructor

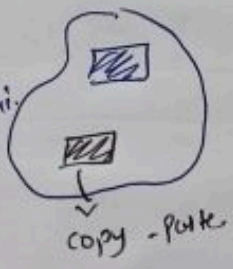
* UML dia



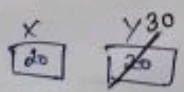
* Shallow copy vs Deep copy

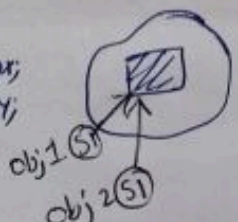
- Same heap mein point karta hai
- Pura copy karke heap mein paste karta hai.

Primitive data ko Sirf copy paste karta

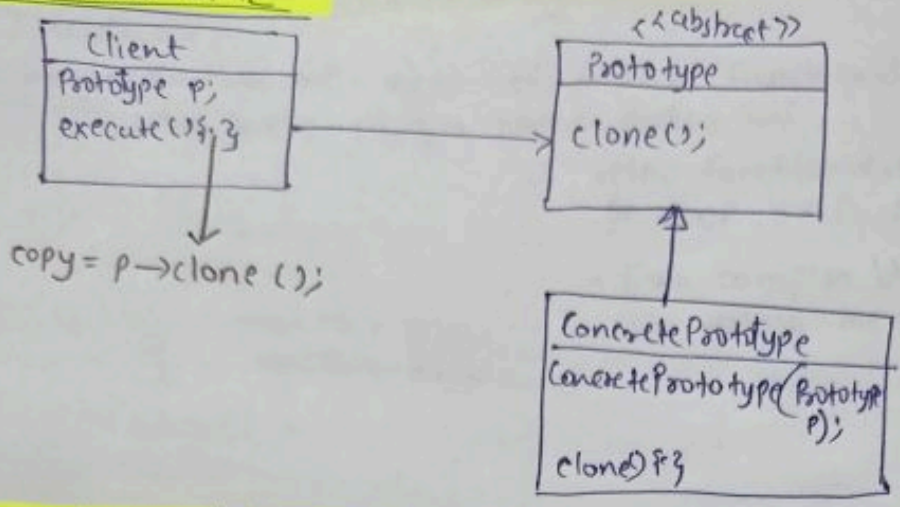



```
class NPC {
    int x, y;
    Student *s1;
    Teacher *t1;
    NPC(NPC *n) {
        this->x = n->x;
        this->y = n->y;
    }
}
```





Standard UML



Standard definition

↳ It let you create new object by copying (cloning) another instance.

Real World Use-case

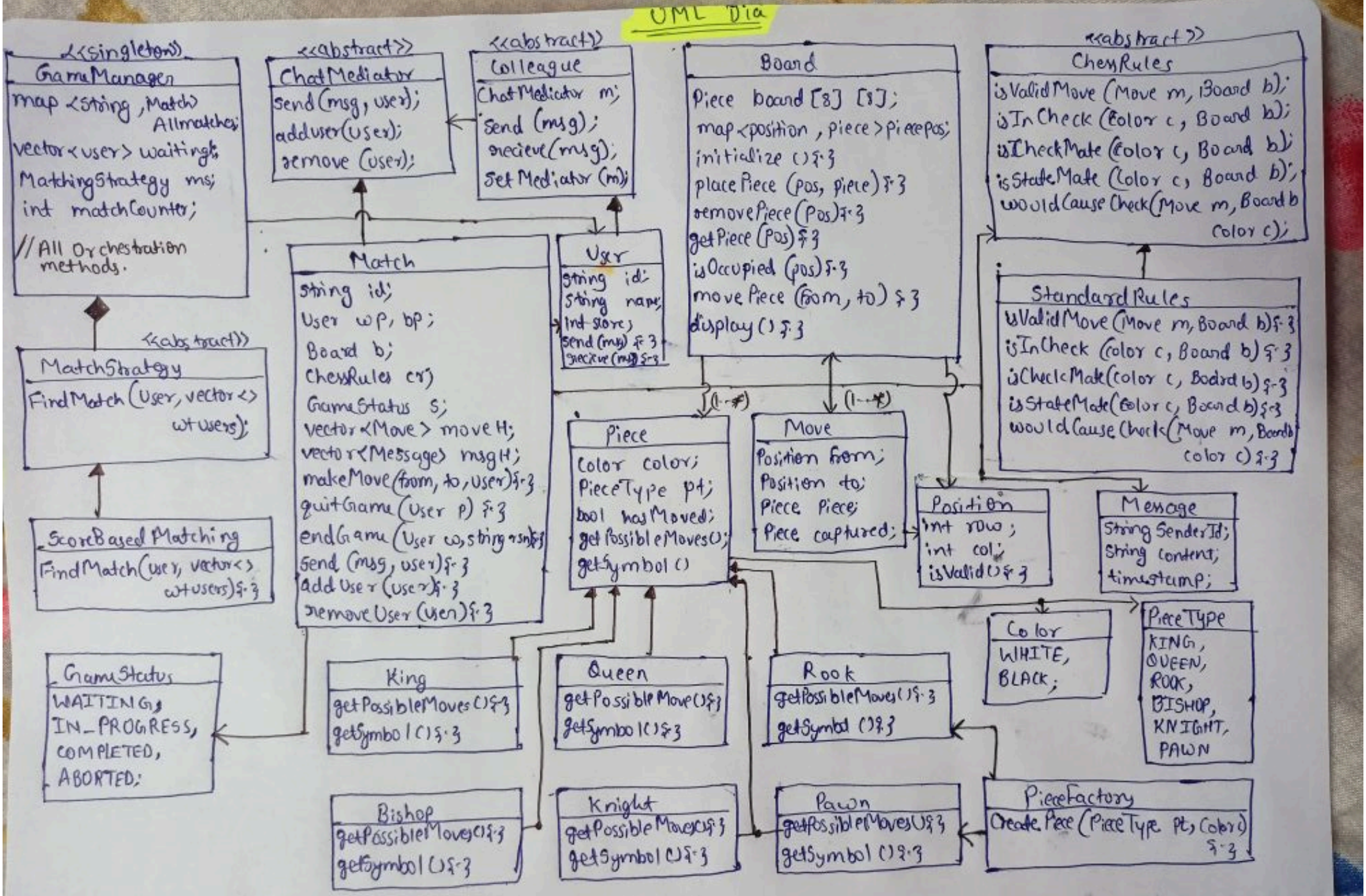
NPC (Non-Player Character)

Build Chess Game

* Requirements

- We can have multiple users playing chess at the same time.
- Score based match making algorithm.
- We have standard chess rules but can be scalable.
- Users within a match can also send/recieve msgs.
- User can quit the match in between.

UML Dia


<<singleton>>
GameManager
map<String, Match> AllMatches;
vector<user> waiting;
MatchingStrategy ms;
int matchCounter;
// All Orchestration methods.
<<abstract>>
ChatMediator
send(msg, user);
addUser(User);
remove(User);
<<abstract>>
Colleague
ChatMediator m;
Send(msg);
recieve(msg);
SetMediator(m);
Board
Piece board[8][8];
map<position, piece> piecePos;
initialize();
placePiece(pos, piece);
removePiece(Pos);
getPiece(Pos);
isOccupied(pos);
movePiece(from, to);
display();
<<abstract>>
ChessRules
isValidMove(Move m, Board b);
isInCheck(Color c, Board b);
isCheckMate(Color c, Board b);
isStateMate(Color c, Board b);
wouldCauseCheck(Move m, Board b, Color c);
Match
string id;
User wp, bp;
Board b;
ChessRules cr;
GameStatus s;
vector<Move> moveH;
vector<Message> msgH;
makeMove(from, to, User);
quitGame(User p);
endGame(User w, string result);
Send(msg, user);
AddUser(user);
removeUser(user);
User
string id;
String name;
int score;
Send(msg);
recieve(msg);
StandardRules
isValidMove(Move m, Board b);
isInCheck(Color c, Board b);
isCheckMate(Color c, Board b);
isStateMate(Color c, Board b);
wouldCauseCheck(Move m, Board b, Color c);
<<abstract>>
MatchStrategy
FindMatch(User, vector<> wtUsers);
ScoreBasedMatching
FindMatch(User, vector<> wtUsers);
Piece
Color color;
PieceType pt;
bool hasMoved;
getPossibleMoves();
getSymbol()
Move
Position from;
Position to;
Piece piece;
Piece captured;
Position
int row;
int col;
isValid();
Message
String SenderId;
String content;
timestamp;
Color
WHITE,
BLACK;
PieceType
KING,
QUEEN,
ROOK,
BISHOP,
KNIGHT,
PAWN
GameStatus
WAITING,
IN_PROGRESS,
COMPLETED,
ABORTED;
King
getPossibleMoves();
getSymbol();
Queen
getPossibleMove();
getSymbol();
Rook
getPossibleMoves();
getSymbol();
Bishop
getPossibleMoves();
getSymbol();
Knight
getPossibleMoves();
getSymbol();
Pawn
getPossibleMoves();
getSymbol();
PieceFactory
Create.Piece(PieceType pt, Color c);
(1-*)
(1-*)

Visitor Design Pattern

* Introduction

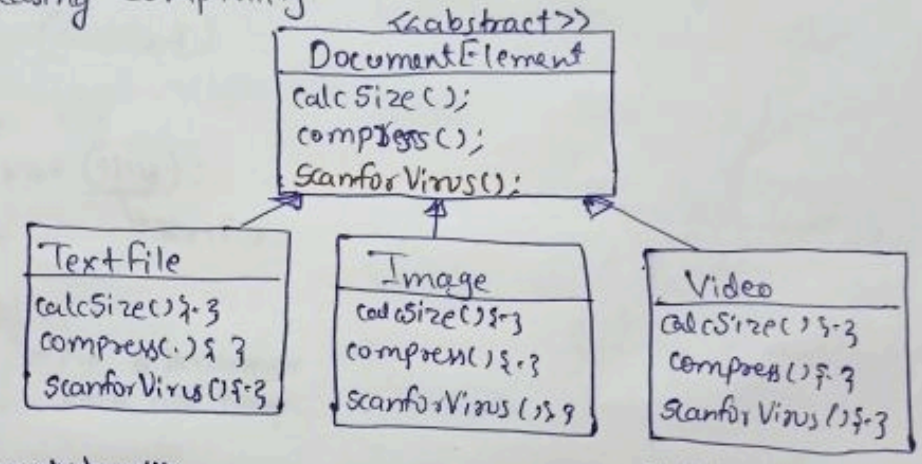
- **Problem:-** Ek class mein agar new feature / functionality add karna hai toh class mein changes karna padta hai.

```
class A {
    m1(){..}
    m2(){}
    m3(){..}  // new
    m4(){}    // new
}
```

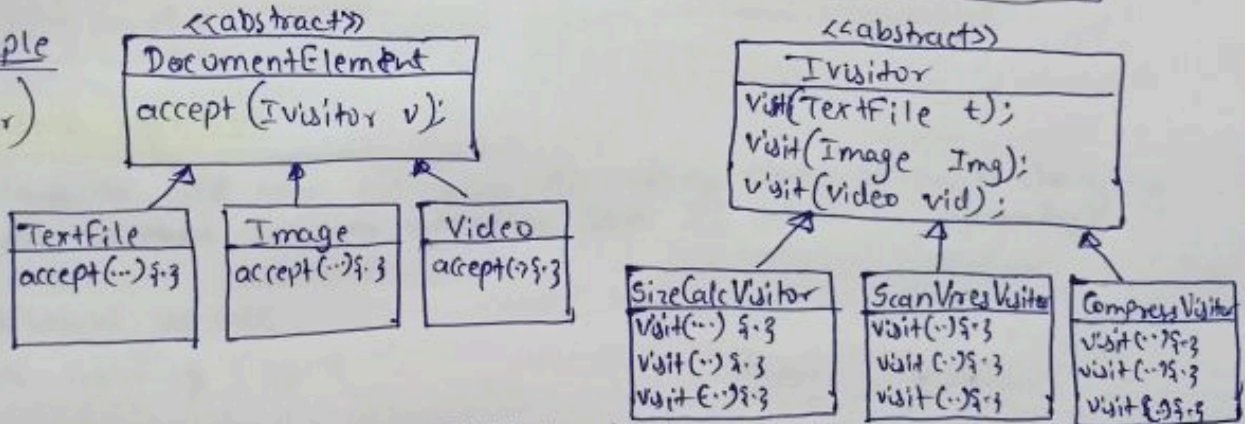
- New functionality add karne ki wajah se OCP, SRP Principal break hota hai.
- Even compiler bhi ho jata hai. Unit testing mein problem aayegi.

- **Solution:-** Visitor

- **Example:-** (Without visitor) DocumentElement mein new features add hote jaa rhe hai. Jisse subclass mein changes aa rhe hai. Breaking OCP, SRP, increasing complexity.



- **Example** (With visitor)

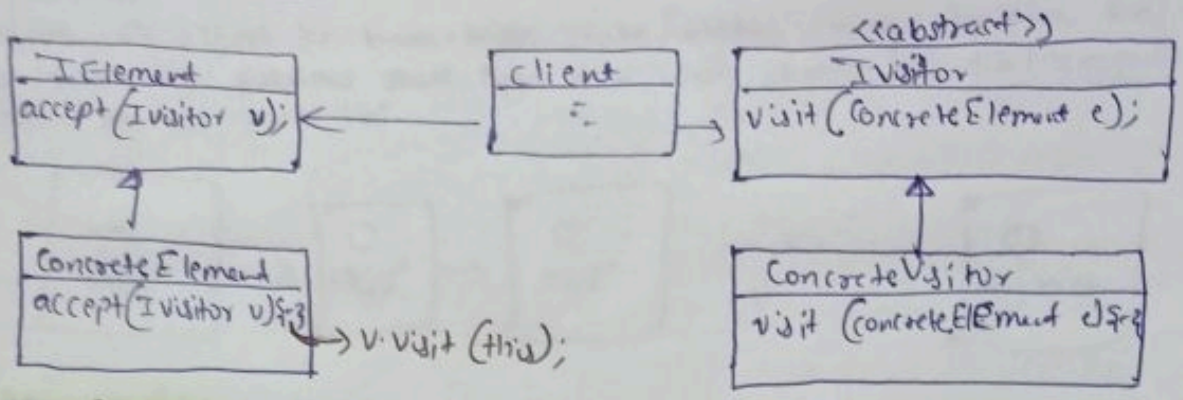


- **logic**
  - Client ke main() mein hum TextFile() ka obj. bana rhe hai. And fir accept ke andar calcSizeVisitor ka constructor bana rhe hai. Matlab humko size calculate karna hai TextFile ka.
  - Ab accept(IVisitor v) mein yeh logic hoga ki visit ko call karo But kis ka visit call hoga woh "this" par depend hai. "this" mein hum TextFile ko daldenge toh jis ka constructor bana hai aur jis ka "this" mein name hai uss ka method call hoga.

```
main(){
    DocumentElement file1 = new TextFile();
    file1.accept(new SizeCalcVisitor());
}
```

```
accept(IVisitor v){
    v->visit(this);
}
```

* Standard UML dia



* Double Dispatch

↳ Do(2) reference milkar decide karte hai ki kiska aur kaunsa method Call hoga.

Ineg, ref 1 :- SizeCalcVisitor() (constructor)

ref 2 :- this (method)

accep(IVisitor v){

v → visit(this);

↳ calcSizeVisitor ↳ TextFile.

}

* Strategy vs Visitor

↳ No. of behaviour same honge
- Uss behaviour ko alag-alag tarike se implement kar sktehe.

↳ No. of behaviour same nhi honge.

Standard Definition

↳ Allows to add new operation to existing classes without changing there structure. Separates operation from the object it operates.

Real World Use-case

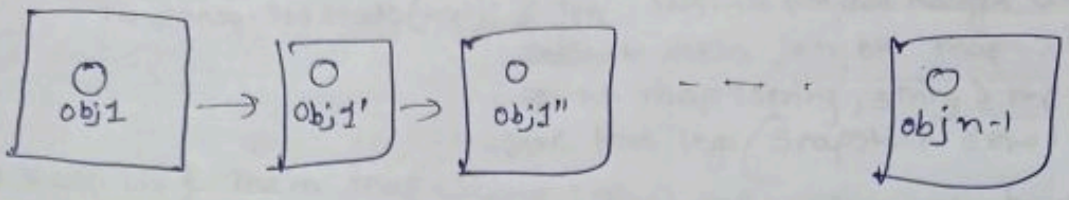
① Game rendring Engine

② Document

Memento Design Pattern

* Introduction

Consider, Ek object ka baar-baar state / status change ho rha hai. Aur hum ko previous state kya tha woh janna hai toh Memento Pattern ka use karte hai.



* 3 keywords

① Originator ⇒ Woh object jis ki state change hoti hai baar-baar. Humare e.g mein obj1. hai.

② Memento ⇒ Haar state ka snapshot hi memento. hai

③ Caretaker ⇒ List of memento hai. Basically DB jaisa kaam karta hai.

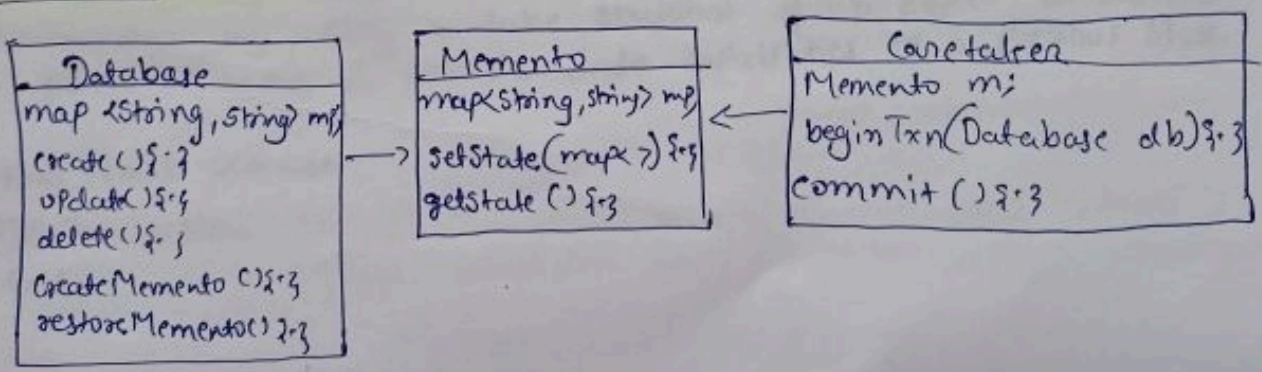
Example: DB Transaction Management

Consider, 1 mein Aditya ko nisha and 101 ko 102 karna tha. Humne transaction ko begin kiya par sirf 102 hi update hua nisha nhi hua update. Toh hum DB ko bolenge ki rollback karo. Agar sab sahi hua toh commit kar denge hum.

```
begin_transaction (){
    ≋
    if (EverythingIsfine) {
            commit();
    else
            rollback();
    }
}
```

1	Aditya	101
2	Rahul	103

* UML dia



- map<String, String> mp;
  - String (1st) → Key
  - String (2nd) → value

- map<String, String> mp; in Database table/class we will consider that this is our db.
- Memento class mein same map hoga.

```
createMemento(){
        memento.setState(mp); // yeh setState ko call karega aur
}                             setState mein jo bhi map<> hoga
                              uss ko map<String, String> mp mein
                              save kar lega (Snapshot done).
```

getState(){} mein map<String, String> mp; mein jo kuch hoga woh return kar dega (matlab ~~map~~ map).

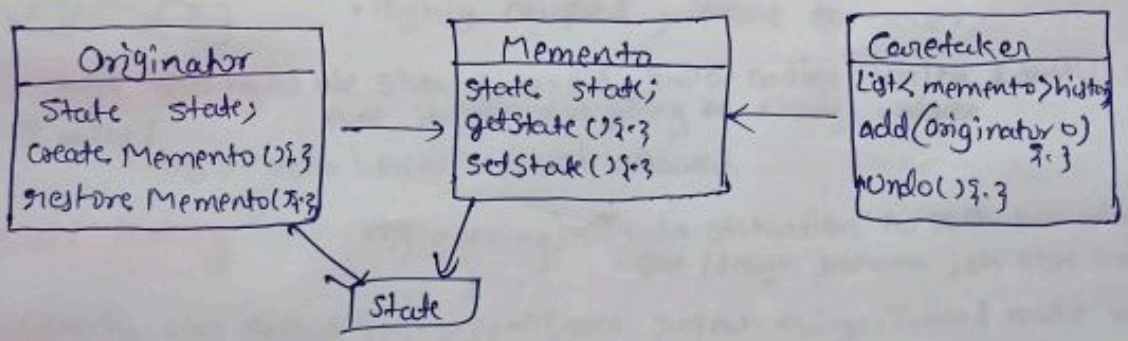
```
restore(Memento m){
        mp = m.getState();
}
```

- m = db.createMemento() ⟷ createMemento(){...} ⟶ setState(){}

Memento m ⟵ store in ⟶ map<String, String> mp

(caretaker)

Standard UML



Standard Def^n :-

↳ Provides an ability to take snapshot of an object at various point in time & provide undo capabilities to a previous state.

Real World use-case

① DB Transaction

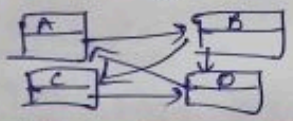
② Git

Anti-Pattern

* Introduction
  - Hum ne dekha ki SOLID principle hote hai. Jis ko humein follow karna hota hai. Par kbhi-kbhi, hum usse break kar dete hai. Isliye humne design patterns dekha. Jis se hum SOLID principle ko naa break kare.
  - Ab anti-pattern woh hai joh hum galati se kar dete hai code karte time (koi bhi chiz).

Types of Anti-Pattern.

1. God Object ⇒ Ek aisi class hai/object hai jis ko humne bhot sari responsility de di toh SRP / OCP ko break kar dega & complex kar dega object ko.
   - Ab bhot sare log sochenge ki orchestrator class God obj hai yaa nhi
     - Agar 'o' class mein sab kaam delicate kar rha hai toh woh nhi hai.
     - Agar 'o' class mein sab kaam declare hai toh ~~to~~ oh hai.
2. Spaghetti code ⇒ Hum ko start & end pata hi nhi hai. Sare object ek-dusre ke upar depend hai. And they form more complex str.
   - Highly coupled, prone to error.



3. Hard-coding ⇒ We should avoid hard-coding unless & untill we know that it will not going to change values.

```
eg. main() {
        String s = "Hello World"; //Not allowed.
    }
```

4. Gold-Plating (over-engineering) ⇒ Jyada situation ko soch kar alag-alag object/design banana joh kbhi hoga hi nhi.
5. DRY (Do not Repeat yourself) ⇒ Don't repeat things. Instead make new class and do it.
6. Constructor Overloading ⇒ We solved by builder design pattern.
7. Over use of Getter/Setter ⇒ G/S help karti hai private memembers ko access karne mein. Par bhot sare log sab ko G/S mein dal dete hai. Toh private memembers ka kuch matlab hi nhi hua.
8. Premature Optimization ⇒ Phele joh banana hai woh bana lo, badd mein optimize karo. Like in DSA ⇒ First brut force, then optimize one
   (make it work, then make it Fast)
9. Inheritance overuse ⇒ subclass mein woh methods ~~mein~~ bhi dalne honge joh used mein nhi aayega.

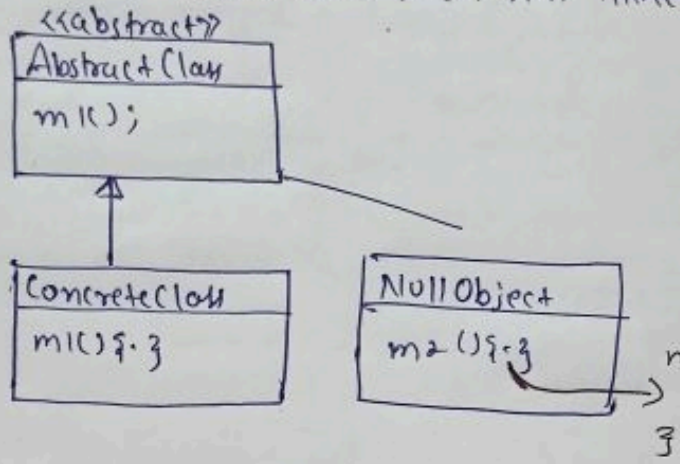
NULL object Pattern

- Aisa situation aa skta hai ki kbhi-kbhi jaha obj Null paas ho jayega toh error aa jayega.
- Error ko avoid karne ke liye hum yeh "if" wali condition likhte hai.

```
if (obj == null)
    obj.m1();
```

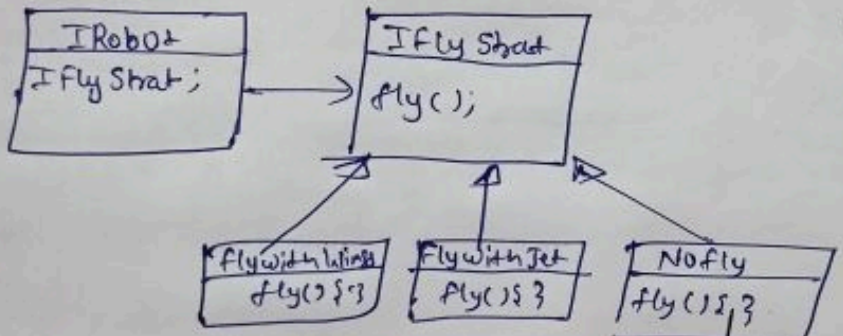
- E.g. mein m1() ko call karne se phele hi null hai yaa nhi check kar rhe hai.
- Ab dussa solution yeh hai ki hum ek abstract class bana le. Aur Null ke liye hum usse concrete class bana le.
- Aur uss Null class mein hum kuch nhi likhenge.

```
class obj {
    ≡
    m1() { }
    m2() { }
}
```



* Example:-

① Strategy design pattern (SDP)



- SDP mein fly method hai. Par kbhi-kbhi client fly nhi karwana chahta hoga robot ko toh woh kuch bhi nhi dalega. Error aa skta hai. Toh hum ne Nofly class bhi banaali.

② Command Design Pattern